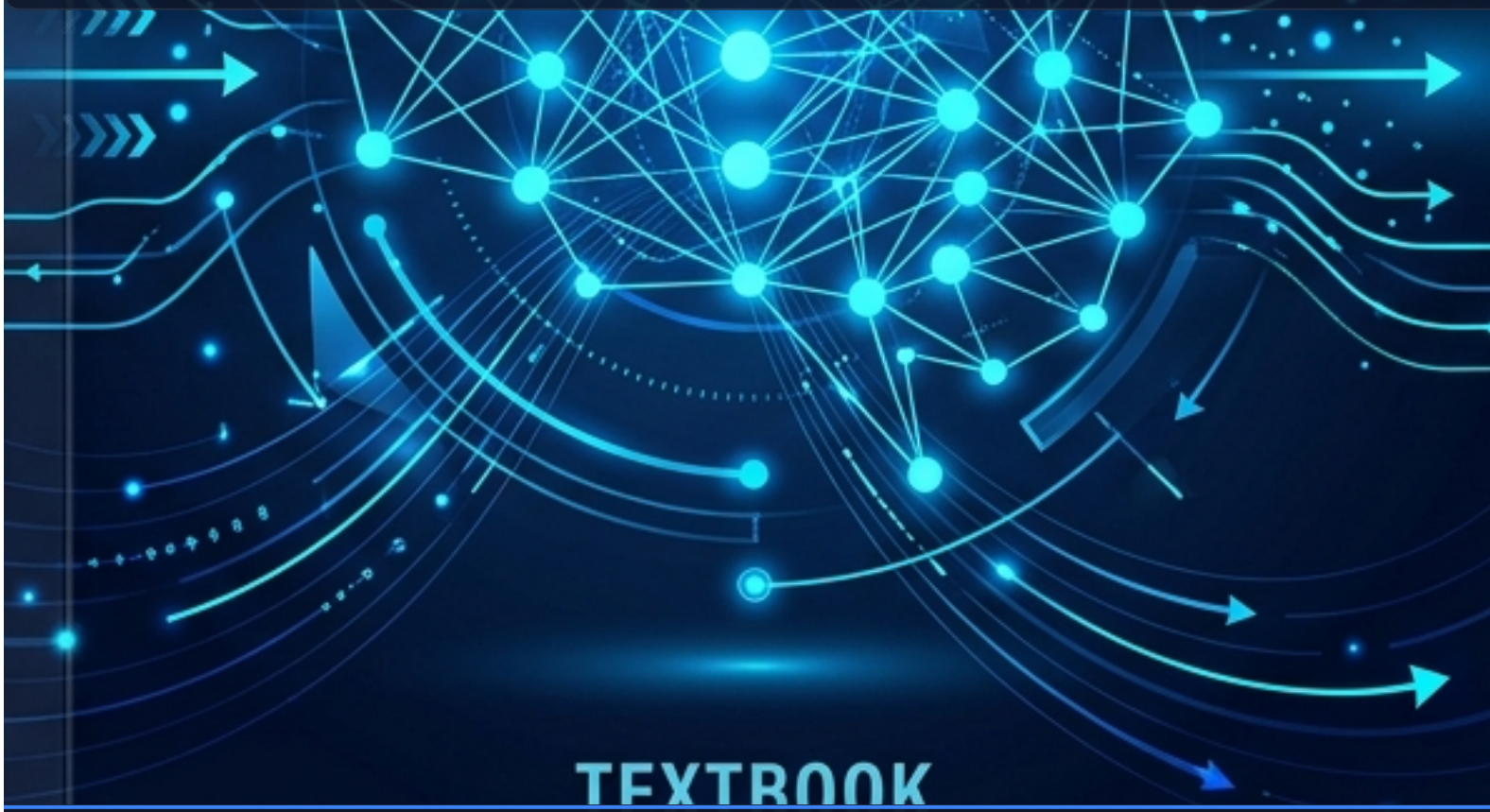
Interactive Machine Learning Curriculum

Theory, Hands-on Exercises & Complete Reference Solutions

A comprehensive, self-paced curriculum spanning 20 interactive chapters. Covers foundational data manipulation, preprocessing, and plotting, classical regression and classification algorithms, unsupervised clustering, feature engineering, time-series forecasting, NLP, recommender systems, and deep learning.



Format: Comprehensive Textbook & Workbook

Chapters: 20 Curriculum Units

Exercises: 60 Interactive Challenges (Easy, Medium, Hard)

Generated on June 27, 2026

Table of Contents

CHAPTER 1

Introduction & Data Manipulation (Pandas & NumPy)

Theory & Foundations

In this notebook, you will explore the building blocks of data science and machine learning in Python: **NumPy** (Numerical Python) and **Pandas** (Python Data Analysis Library). Almost every machine learning pipeline starts with loading, manipulating, and understanding raw data.

1. NumPy: Vectorization and Arrays

NumPy provides the `ndarray` object, a multidimensional array that is memory-efficient and supports fast vector mathematical operations. - **Vectorization**: Instead of writing nested `for` loops in Python, we perform operations directly on arrays. - **Slicing and Indexing**: Just like Python lists, but extends to multiple dimensions.

2. Pandas: DataFrames and Series

Pandas is built on top of NumPy and provides: - **Series**: A 1D array with labeled axes. - **DataFrame**: A 2D labeled data structure with columns of potentially different types (similar to a SQL table). - **Data Selection**: `loc` (label-based) and `iloc` (integer-based). - **Groupby and Aggregate**: Grouping records by one or more keys and running statistical aggregations (mean, sum, count, etc.).

Deep Dive: Vectorization and Memory Alignment

In numerical computing, **vectorization** replaces explicit loop constructs with array expressions. NumPy enables vectorization by implementing computations in optimized, compiled C code. Under the hood, a NumPy `ndarray` is a contiguous block of memory. Because elements are stored contiguously, modern CPUs can leverage **SIMD** (Single Instruction, Multiple Data) instructions to process multiple operations in a single CPU cycle. In Pandas, data alignment is a fundamental behavior: when operations are performed on two Series or DataFrames, Pandas automatically aligns elements based on index labels. If labels do not overlap, Pandas inserts `NaN` to represent the mismatch, preserving data integrity.

Example Implementation

```
import numpy as np
import pandas as pd

# NumPy Demonstration
arr = np.array([1, 2, 3, 4, 5])
print("Original NumPy array:", arr)
print("Vectorized multiplication (arr * 2):", arr * 2)

# Pandas Demonstration
data = {
    'Product': ['Apple', 'Banana', 'Orange', 'Apple', 'Banana'],
    'Sales': [100, 150, 80, 120, 90]
}
df = pd.DataFrame(data)
print("Pandas DataFrame:")
print(df)

# Groupby example
grouped = df.groupby('Product').sum()
print("Sum of Sales by Product:")
print(grouped)
```

EXECUTION OUTPUT

```
Original NumPy array: [1 2 3 4 5]
Vectorized multiplication (arr * 2): [ 2  4  6  8 10]
Pandas DataFrame:
  Product  Sales
0   Apple    100
1  Banana    150
2  Orange     80
3   Apple    120
4  Banana     90
Sum of Sales by Product:
         Sales
Product
Apple      220
Banana     240
Orange      80
```

Exercises

Now it is your turn! Run the verification code cell after each exercise to check your answers.

EASY

Exercise 1: Filter and Select

Given a dataset of employees, write a Pandas query to select only the **Name** and **Score** of employees who are strictly older than **30** (Age > 30). Store your output in a DataFrame named `result_df`.

How to solve: Use Pandas comparison operators (e.g. `df['Age'] > 30`) to generate a boolean mask. Apply this mask to the DataFrame to filter rows, then index using a list of column names: `df[mask][['Name', 'Score']]`.

TEMPLATE CODE:

```
# Define the initial DataFrame
df = pd.DataFrame({
    'Name': ['Alice', 'Bob', 'Charlie', 'David'],
    'Age': [25, 30, 35, 40],
    'Score': [85, 90, 95, 80]
})

# TODO: Filter rows where Age > 30 and select columns 'Name' and 'Score'
result_df = ____

print(result_df)
```

MEDIUM

Exercise 2: Groupby and Aggregations

You have a DataFrame containing product sales transactions. Group the sales by the **Category** column and calculate: 1. The **average** (mean) of `Revenue`. 2. The **sum** of `Quantity`.

Name the columns in the aggregated DataFrame exactly `Avg_Revenue` and `Total_Quantity`. Store your output in a DataFrame named `agg_df`.

How to solve: Call `.groupby('Category')` on the DataFrame. Chain the `.agg()` method, passing named arguments to rename columns and specify the aggregation function: `Avg_Revenue=('Revenue', 'mean'), Total_Quantity=('Quantity', 'sum')`.

TEMPLATE CODE:

```
sales_df = pd.DataFrame({
    'Category': ['Electronics', 'Clothing', 'Electronics', 'Home',
'Clothing', 'Home'],
    'Revenue': [1000.0, 200.0, 1500.0, 300.0, 400.0, 600.0],
    'Quantity': [10, 5, 15, 3, 8, 6]
})

# TODO: Group by 'Category' and aggregate 'Revenue' with mean and 'Quantity'
with sum.
# Rename columns to 'Avg_Revenue' and 'Total_Quantity'.
agg_df = _____

print(agg_df)
```

HARD

Exercise 3: Matrix Row Normalization and Multiplication

Write a function `normalize_and_multiply(A, B)` using NumPy that: 1. Normalizes a 2D numpy array A along rows, so that each row sums to 1. 2. Performs matrix multiplication of the normalized A with a 2D numpy array B.

Formula: For a row i, the normalized row elements are:

$$A_{norm}[i, j] = (A[i, j])/(\sum_k A[i, k])$$

Then compute $C = A_{norm} \cdot B$.

How to solve: First, sum elements along rows using `np.sum(A, axis=1, keepdims=True)`. The `keepdims=True` argument is crucial to allow broadcasting division. Divide A by this sum to get the normalized matrix, then perform matrix multiplication using the `@` operator: `A_norm @ B`.

TEMPLATE CODE:

```
def normalize_and_multiply(A, B):

# TODO: Normalize rows of A to sum to 1, then return A_norm multiplied by B.
    pass

# Test values
A = np.array([[1.0, 2.0], [3.0, 4.0]])
B = np.array([[5.0], [6.0]])
print("Normalized & Multiplied:
", normalize_and_multiply(A, B))
```

CHAPTER 2

Data Cleaning & Preprocessing

Theory & Foundations

Machine learning algorithms require clean, structured numerical inputs. In raw data, this is rarely the case. In this notebook, we will explore: 1. **Handling Missing Values**: Imputation (filling in missing cells) or removal. 2. **Categorical Encoding**: Converting strings into numerical representations using Ordinal Encoding or One-Hot Encoding. 3. **Feature Scaling**: Centering and scaling distributions using Min-Max scaling or Standardization (Z-score).

1. Missing Values

Common strategies: - Fill with constant (e.g. 0). - Fill with statistics (mean, median, mode). - Predictive imputation (using other features).

2. Encoding Categorical Variables

- **Ordinal Encoding**: Converting categorical labels to integers (0, 1, 2...) when there is an inherent ordering (e.g., Small, Medium, Large).
- **One-Hot Encoding**: Creating binary dummy columns for each category (e.g., Color_Red, Color_Blue) when there is no order.

Deep Dive: Data Integrity and Encoding Trade-offs

Data cleaning is the most critical phase in a machine learning project. When dealing with missing values, **imputation** (filling in gaps) is preferred over removing rows to retain sample size. However, imputing with a simple statistic like the mean can artificially reduce variance. For categorical features, the choice of encoding affects how a model interprets the feature space: - **Ordinal Encoding**: Used when categories have a natural sequence (e.g., small, medium, large). It maps values directly to ordered integers. - **One-Hot Encoding**: Used when categories are nominal (unordered, e.g., colors). It creates dummy columns. Using ordinal encoding on nominal variables is a common mistake; it forces the model to assume a linear relationship where none exists.

Example Implementation

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import OneHotEncoder, MinMaxScaler

# Example of imputation
s = pd.Series([1, np.nan, 3, np.nan, 5])
print("Imputed with mean:", s.fillna(s.mean()).tolist())

# Example of One-Hot Encoding
df_cat = pd.DataFrame({'Color': ['Red', 'Blue', 'Red']})
onehot = pd.get_dummies(df_cat, columns=['Color'])
print("
One-hot Encoded:")
print(onehot)
```

Exercises

EASY

Exercise 1: Missing Value Imputation

Impute the missing values (`NaN`) in the series `s` below using the **mean** value of the series. Store the output in a variable `s_imputed`.

How to solve: Use the `fillna()` method on the series `s`. Calculate the mean of the series using `s.mean()` and pass it as the argument to fill all missing (NaN) values.

TEMPLATE CODE:

```
s = pd.Series([10.0, np.nan, 20.0, 30.0, np.nan, 40.0])

# TODO: Impute missing values with mean
s_imputed = ____

print(s_imputed)
```

MEDIUM

Exercise 2: Ordinal & One-Hot Encoding

Given a DataFrame `df` containing a categorical column `'Size'` with values `['Small', 'Medium', 'Large', 'Medium', 'Small']` and a `'Color'` column. 1. Perform **Ordinal Encoding** on the `'Size'` column. Map `'Small'` to `0`, `'Medium'` to `1`, and `'Large'` to `2`. Store this in a new column `'Size_Encoded'`. 2. Perform **One-Hot Encoding** on the `'Color'` column. Use Pandas `pd.get_dummies()` and ensure the resulting columns prefix is `'Color'`. Keep all original columns (except `'Color'`, which get_dummies replaces automatically). Store your final DataFrame as `encoded_df`.

How to solve: Use the `map()` method with a dictionary mapping categories to numbers for the ordinal column: `df['Size'].map({'Small': 0, 'Medium': 1, 'Large': 2})`. For nominal encoding, use `pd.get_dummies(df, columns=['Color'])`.

TEMPLATE CODE:

```
df = pd.DataFrame({
    'Size': ['Small', 'Medium', 'Large', 'Medium', 'Small'],
    'Color': ['Red', 'Blue', 'Red', 'Green', 'Blue']
})

# TODO: Perform Ordinal encoding on 'Size' (0, 1, 2)
# and One-Hot encoding on 'Color'.
encoded_df = _____

print(encoded_df)
```

HARD

Exercise 3: Custom Scaling with Percentile Clipping

Outliers can severely affect machine learning models. Write a function `custom_scale(arr, min_val, max_val)` that: 1. Identifies the **10th percentile** (p_{10}) and **90th percentile** (p_{90}) of the 1D input array `arr`. 2. **Clips** any values below p_{10} to p_{10}, and any values above p_{90} to p_{90}. 3. **Scales** the clipped array linearly into the range $[min\_val, max\_val]$.

Formulas: Min-max scaling to range $[a, b]$:

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}} \cdot (b - a) + a$$

How to solve: Find the 10th and 90th percentiles of the array using `np.percentile(arr, 10)` and `np.percentile(arr, 90)`. Use `np.clip(arr, p10, p90)` to clip values, then apply Min-Max scaling: `(clipped - clipped.min()) / (clipped.max() - clipped.min()) * (max_val - min_val) + min_val`.

TEMPLATE CODE:

```
def custom_scale(arr, min_val, max_val):
    # TODO: Implement 10/90 percentile clipping and min-max scaling to [min_val, max_val]
    pass

# Test custom scaling
arr = np.array([-100, 1, 2, 3, 4, 5, 6, 7, 8, 100])
print("Scaled Array:", custom_scale(arr, 0.0, 1.0))
```

CHAPTER 3

Data Visualization & Plotting

Theory & Foundations

Data visualization is key to exploratory data analysis (EDA). It allows you to: - Identify patterns and distributions. - Discover relationships between variables. - Detect outliers and anomalies.

In Python, the primary plotting libraries are: 1. **Matplotlib**: A low-level plotting library providing full control over the figure, axes, labels, and ticks. 2. **Seaborn**: A high-level data visualization library built on top of Matplotlib. It provides visually appealing default styles and handles integration with Pandas DataFrames natively.

Deep Dive: Visual Analytics and Correlation

Data visualization is an exploratory tool that reveals structural patterns, anomalies, and relationships. Histograms plot the frequency distribution of continuous variables, showing skewness, modality (unimodal, bimodal), and potential outliers. Scatter plots visualize bivariate relationships, allowing us to eyeball correlations and linearities. The Pearson correlation coefficient r measures the linear relationship between variables, ranging from -1 to $+1$. A correlation matrix heat map helps identify **multicollinearity** (high correlation between feature variables), which can destabilize linear models and make coefficient interpretations unreliable.

Example Implementation

```
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
import pandas as pd

# Set plot style
sns.set_theme(style="darkgrid")

# Create sample data
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Generate demonstration plot
plt.figure(figsize=(8, 4))
plt.plot(x, y, label="Sin(x)", color="purple", lw=2)
plt.title("Matplotlib Demo Line Plot")
plt.xlabel("X")
plt.ylabel("Y")
plt.legend()
plt.show()
```

Exercises

EASY

Exercise 1: Plotting a Distribution Histogram

Plot a histogram of the provided data array. You must: 1. Set the number of bins to `20`. 2. Add a title `'Feature Distribution'`. 3. Label the X-axis as `'Value'` and the Y-axis as `'Frequency'`. 4. Turn on the grid. 5. Display the plot.

How to solve: Use Matplotlib's `plt.hist(data, bins=20)`. Add labels and titles using `plt.xlabel('Value')`, `plt.ylabel('Frequency')`, and `plt.title('Feature Distribution')`. Call `plt.show()` to display.

TEMPLATE CODE:

```
np.random.seed(42)
data = np.random.normal(loc=0.0, scale=1.0, size=500)

# TODO: Plot a histogram of 'data' using Matplotlib/Seaborn and set title, labels, and grid.
```

MEDIUM

Exercise 2: Scatter Plot with Regression Line and Hue

Using the provided `tips` dataset, plot a scatter plot of `total_bill` (X-axis) vs `tip` (Y-axis) colored by the `smoker` column. You must include a regression line for each group. *Hint*: Seaborn's `lmplot` or `regplot` can do this.

How to solve: Use Seaborn's `sns.lmplot(x='x', y='y', hue='group', data=df)`. The `lmplot` function automatically fits and plots a linear regression line for each group defined by the `hue` column.

TEMPLATE CODE:

```
# Load standard dataset
tips = sns.load_dataset("tips")

# TODO: Create a scatter plot of 'total_bill' vs 'tip' colored by 'smoker'
with a regression line.
# Note: Use sns.lmplot or equivalent to build this visualization.
```

HARD

Exercise 3: Filtered Correlation Heatmap

When dealing with many numerical features, correlation heatmaps are extremely helpful. In this exercise: 1. Calculate the correlation matrix of the numerical features in the provided dataset. 2. Filter the correlations so that any correlation coefficients with an absolute value strictly less than `0.3` are masked or set to `NaN` (to reduce visual noise). 3. Plot the correlation matrix as a Seaborn heatmap, with values annotated (`annot=True`) and using the `'coolwarm'` colormap.

How to solve: Compute the correlation matrix with `corr = df.corr()`. Filter weak correlations by setting absolute values less than 0.3 to NaN: `corr[corr.abs() < 0.3] = np.nan`. Plot using `sns.heatmap(corr, annot=True, cmap='coolwarm')`.

TEMPLATE CODE:

```
# Generate synthetic dataset with correlated columns
np.random.seed(42)
X = np.random.randn(100, 5)
# Add some correlation
X[:, 1] = X[:, 0] * 0.8 + np.random.randn(100) * 0.6
X[:, 2] = -X[:, 0] * 0.5 + np.random.randn(100) * 0.8
df = pd.DataFrame(X, columns=['feat_A', 'feat_B', 'feat_C', 'feat_D', 'feat_E
'])

# TODO: Calculate correlation, filter out correlations with absolute value <
0.3, and plot heatmap.
```

CHAPTER 4

Regression (Linear & Logistic)

Theory & Foundations

Regression is a supervised learning task where the model predicts continuous target outputs. In this notebook, we cover: 1. **Simple and Multiple Linear Regression**: Minimizing Mean Squared Error (MSE). 2. **Gradient Descent Optimization**: How parameters are learned iteratively. 3. **Logistic Regression**: Used for classification, predicting probabilities using the Sigmoid activation function:

$$\sigma(z) = (1)/(1 + e^{-z)}$$

1. **Regularization**: L_1 regularization (Lasso, which drives some weights to exactly 0) and L_2 regularization (Ridge, which shrinks weights).

Deep Dive: Loss Optimization and Sparse Regularization

Linear Regression fits a hyperplane that minimizes the Mean Squared Error (MSE):

$$L = (1)/(N) \sum_{i=1}^{N} (y_i - \mathbf{w}^T \mathbf{x}_i - b)^2$$

Logistic Regression extends this to binary classification by passing the linear output through the Sigmoid activation:

$$P(y=1|x) = \sigma(\mathbf{w}^T \mathbf{x} + b) = (1)/(1 + e^{-(\mathbf{w}^T \mathbf{x} + b))}$$

To fit these models, **Gradient Descent** iteratively calculates partial derivatives of the loss function with respect to weights, updating them in the direction of steepest descent. When models overfit, **regularization** penalizes large weights: - **L1 (Lasso)**: Adds an absolute weight penalty ($||w||_1$). This results in **sparsity**, setting less important feature weights to exactly zero, thus acting as feature selection. - **L2 (Ridge)**: Adds a squared weight penalty ($||w||_2^2$). It shrinks weights close to zero but keeps all features.

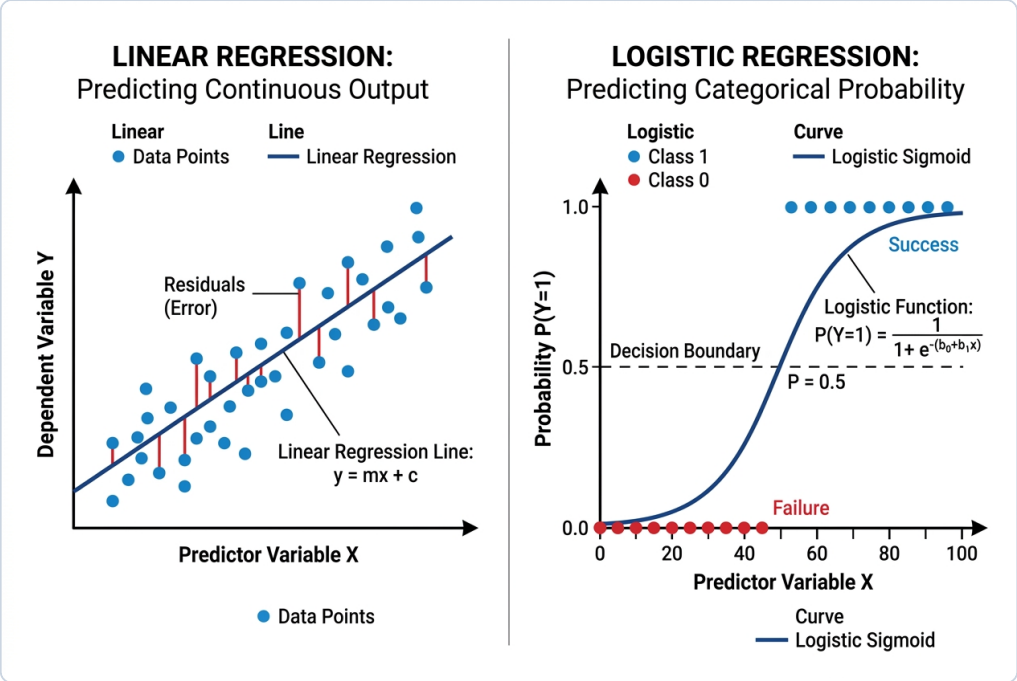


Figure 4.1: Linear Regression fitting vs. Logistic Regression Sigmoid curve.

Example Implementation

```
import numpy as np
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import mean_squared_error

# Quick scikit-learn regression example
X = np.array([[1], [2], [3], [4]])
y = np.array([2.1, 3.9, 6.1, 8.0])

reg = LinearRegression().fit(X, y)
print("Coefficients:", reg.coef_)
print("Intercept:", reg.intercept_)
```

Exercises

EASY

Exercise 1: Fit a Linear Regression Model

Fit a `LinearRegression` model on the training features `X_train` and target `y_train` below. Store your fitted model object in a variable called `model`. Then predict the outputs on the test set `X_test` and store them in a variable `preds`.

How to solve: Import `LinearRegression` from `sklearn.linear_model`. Instantiate it, fit the model on training data using `.fit(X_train, y_train)`, and make predictions on the test set using `.predict(X_test)`.

TEMPLATE CODE:

```
np.random.seed(42)
X_train = np.random.rand(100, 1) * 10
y_train = (2.5 * X_train + 4 + np.random.randn(100, 1)).ravel()

X_test = np.array([[2.0], [5.0], [8.0]])

# TODO: Fit the model and compute predictions
model = _____
preds = _____

print("Predictions:", preds)
```

MEDIUM

Exercise 2: Gradient Descent from Scratch

Implement gradient descent for a single-variable linear model $y = w \cdot x + b$. Given training features X, targets y, learning rate lr, and training epochs, write a function `gradient_descent(X, y, lr, epochs)` that: 1. Initializes $w = 0.0$ and $b = 0.0$. 2. For each epoch, computes the predictions $\hat{y} = w \cdot x + b$. 3. Computes the partial derivatives (gradients) of the MSE loss:

$$(\partial L)/(\partial w) = -(2)/(N) \sum_{i=1}^{N} x_i (y_i - \hat{y}_i)$$

$$(\partial L)/(\partial b) = -(2)/(N) \sum_{i=1}^{N} (y_i - \hat{y}_i)$$

1. Updates w and b using gradient descent step:

$$w \leftarrow w - lr \cdot (\partial L)/(\partial w)$$

$$b \leftarrow b - lr \cdot (\partial L)/(\partial b)$$

1. Returns the learned parameters `(w, b)`.

How to solve: Initialize w and b to 0.0. In a loop over epochs, calculate predictions: `pred = w * X + b`. Calculate gradients: `dw = -2/N * np.sum(X * (y - pred))` and `db = -2/N * np.sum(y - pred)`. Update weights: `w -= lr * dw` and `b -= lr * db`.

TEMPLATE CODE:

```
def gradient_descent(X, y, lr, epochs):
    # TODO: Implement gradient descent optimization
    w, b = 0.0, 0.0
    return w, b

# Test gradient descent
X_gd = np.array([1.0, 2.0, 3.0, 4.0])
y_gd = np.array([3.0, 5.0, 7.0, 9.0])
w_learned, b_learned = gradient_descent(X_gd, y_gd, lr=0.01, epochs=1000)
print(f"Learned parameters: w={w_learned:.4f}, b={b_learned:.4f}")
```

HARD

Exercise 3: Regularization Comparison

Fit two Logistic Regression models on the training data `(X_train, y_train)`: 1. A Logistic Regression model using **L1 penalty** (Lasso) and regularization strength parameter `C=0.1`. (Hint: use `solver='liblinear'`). 2. A Logistic Regression model using **L2 penalty** (Ridge) and regularization strength parameter `C=0.1`.

Extract the coefficients of the features for both models and store them in `coef_l1` and `coef_l2`. Compare the number of zero coefficients.

How to solve: Train `LogisticRegression(penalty='l1', solver='liblinear', C=0.1)` for L1 and `LogisticRegression(penalty='l2', C=0.1)` for L2. Extract coefficients using `model.coef_[0]`.

TEMPLATE CODE:

```
from sklearn.datasets import make_classification
X_train, y_train = make_classification(n_samples=200, n_features=20, n_inform
ative=5, random_state=42)

# TODO: Fit L1 and L2 regularized Logistic Regression models.
# Store the coefficients in coef_l1 and coef_l2 respectively.
coef_l1 = ____
coef_l2 = ____

print("L1 Coefficients:
", coef_l1)
print("
L2 Coefficients:
", coef_l2)
```

CHAPTER 5

Decision Trees & Random Forests

Theory & Foundations

Decision Trees and Random Forests are widely used non-parametric models for classification and regression.

1. Decision Trees

A Decision Tree splits the data recursively into sub-nodes. - **Split Criterion**: Calculated using Gini Impurity or Entropy (for classification), or Variance Reduction (for regression). - **Overfitting**: Trees can easily grow to fit noise in the training set. We control this using constraints like `max_depth`, `min_samples_split`, or `min_samples_leaf`.

2. Random Forests

An ensemble model that builds a forest of randomized decision trees. - **Bagging (Bootstrap Aggregating)**: Each tree is trained on a random bootstrap sample of the training data. - **Feature Randomization**: At each split point, only a random subset of features is considered. - **Averaging**: The final prediction is a vote (classification) or average (regression) of all tree outputs.

Deep Dive: Ensemble Synergy & Bias-Variance Trade-off

Decision Trees partition the feature space recursively by choosing splits that maximize information gain or minimize **Gini Impurity**:

$$\text{Gini} = 1 - \sum_{k=1}^{C} p_k^2$$

Deep trees partition the space too specifically, leading to low bias but high variance (overfitting). Constraining `max_depth` limits the splits to create more general rules. **Random Forests** solve the high variance of individual trees by building an ensemble of diverse trees. Using **Bootstrap Aggregating (Bagging)**, each tree is trained on a random sample of the data (with replacement), and splits are evaluated on a random subset of features. The trees vote (classification) or average (regression) to yield a model with low variance.

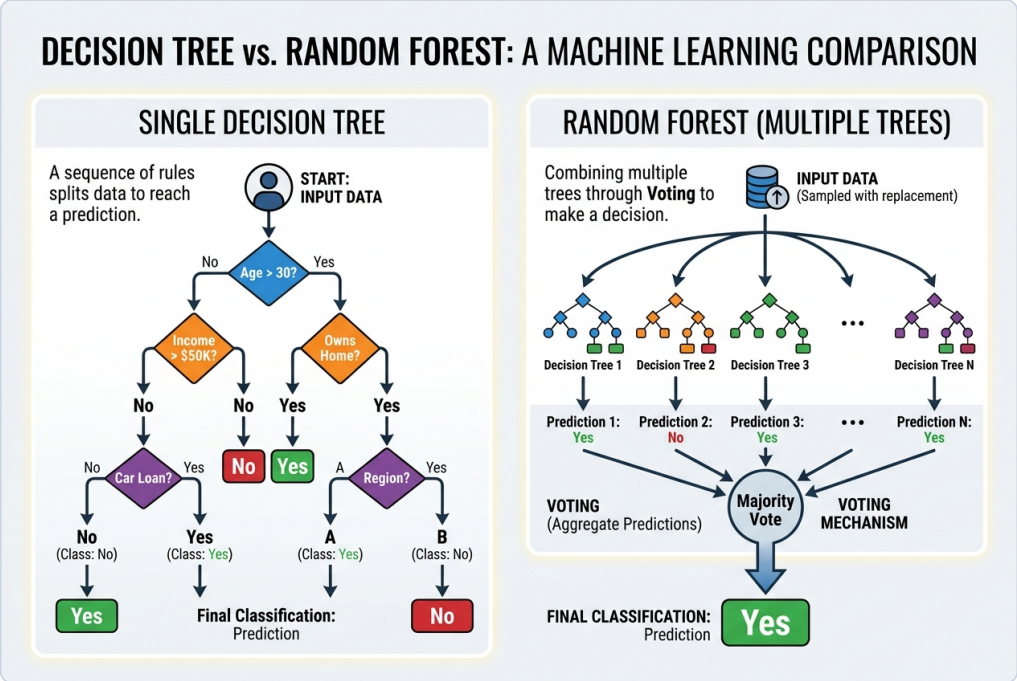


Figure 5.1: Structure of a Single Decision Tree vs. Random Forest Voting.

Example Implementation

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Demonstration
iris = load_iris()
X_tr, X_te, y_tr, y_te = train_test_split(iris.data, iris.target, test_size=0.2, r
andom_state=42)
dt = DecisionTreeClassifier(max_depth=2, random_state=42).fit(X_tr, y_tr)
print("Decision Tree Train Acc:", dt.score(X_tr, y_tr))
print("Decision Tree Test Acc:", dt.score(X_te, y_te))
```

Exercises

EASY

Exercise 1: Train a Constrained Decision Tree

Train a `DecisionTreeClassifier` on features `X_train` and labels `y_train` below. Use a `max_depth` of `3` and set `random_state` to `42`. Store the fitted model object in `dt_model`.

How to solve: Import `DecisionTreeClassifier` from `sklearn.tree`. Instantiate it setting `max_depth=3` and `random_state=42`, then call `.fit(X_train, y_train)`.

TEMPLATE CODE:

```
from sklearn.datasets import make_classification
X, y = make_classification(n_samples=1000, n_features=10, n_classes=2, random_
state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
dom_state=42)

# TODO: Train dt_model
dt_model = _____
```

MEDIUM

Exercise 2: Random Forest Ensemble Tuning

Train a `RandomForestClassifier` on the same training data. Configure it with: 1. `n_estimators=100` 2. `max_depth=5` 3. `min_samples_split=4` 4. `random_state=42`

Store the fitted model in `rf_model`. Compute the accuracy score of your model on the test data `X_test` and store it in `rf_acc`.

How to solve: Train `RandomForestClassifier(n_estimators=100, max_depth=5, min_samples_split=4, random_state=42)`. Call `.predict(X_test)` and calculate the accuracy score using `accuracy_score(y_test, predictions)`.

TEMPLATE CODE:

```
from sklearn.metrics import accuracy_score

# TODO: Train rf_model and compute rf_acc on X_test
rf_model = ____
rf_acc = ____

print("Random Forest Accuracy:", rf_acc)
```

HARD

Exercise 3: Feature Importances & Selection

Tree-based models rank features based on how much they reduce node impurity. In this exercise: 1. Extract the feature importances from your trained `rf_model` and store them in an array called `importances`. 2. Find the index numbers of all features with an importance strictly greater than `0.1`. Store these indices in a numpy array named `selected_indices`. 3. Filter the training dataset `X_train` to include only those selected features. Store this filtered dataset in a variable named `X_train_filtered`.

How to solve: Get feature importances using `rf_model.feature_importances_`. Find indices where importance is greater than 0.1 using `np.where(importances > 0.1)[0]`, and filter training columns: `X_train[:, selected_indices]`.

TEMPLATE CODE:

```
# TODO: Extract importances, find selected_indices, and filter X_train
importances = ____
selected_indices = ____
X_train_filtered = ____

print("Selected feature indices:", selected_indices)
print("Filtered X_train shape:", X_train_filtered.shape)
```

CHAPTER 6

Support Vector Machines & Naive Bayes

Theory & Foundations

In this notebook, we cover two classical and powerful classification algorithms:

1. Support Vector Machines (SVM)

SVM aims to find the optimal hyperplane that separates classes with the maximum margin. - **Support Vectors**: The data points closest to the decision boundary. - **Kernel Trick**: Projects data into a higher-dimensional space to make it linearly separable. Common kernels: Linear, Polynomial, and Radial Basis Function (RBF). - **Hyperparameters**: - `C`: Controls the trade-off between maximizing margin and minimizing classification errors.

2. Naive Bayes

Naive Bayes is a probabilistic classifier based on Bayes' Theorem:

$$P(y \mid x_1 ... x_n) = (P(y)\ P(x_1 ... x_n \mid y))/(P(x_1 ... x_n))$$

It makes the **"naive" assumption** that features are conditionally independent given the class label. Very popular for text classification and spam detection.

Deep Dive: Maximum Margin Separation & Naive Bayes Probabilities

Support Vector Machines (SVM) find the optimal hyperplane that separates classes while maximizing the **margin**—the distance between the boundary and the closest data points (the support vectors). For non-linear datasets, SVMs project data into a higher-dimensional space where classes are linearly separable. This is done using a **Kernel function** (e.g., Radial Basis Function, RBF) without explicitly computing high-dimensional coordinates, saving computational resources. **Naive Bayes** is a probabilistic classifier based on Bayes' Theorem:

$$P(y|x) = (P(x|y)\ P(y))/(P(x))$$

It is called "naive" because it assumes all features are conditionally independent given the class label. Despite this simplification, it is highly effective and fast for high-dimensional text datasets.

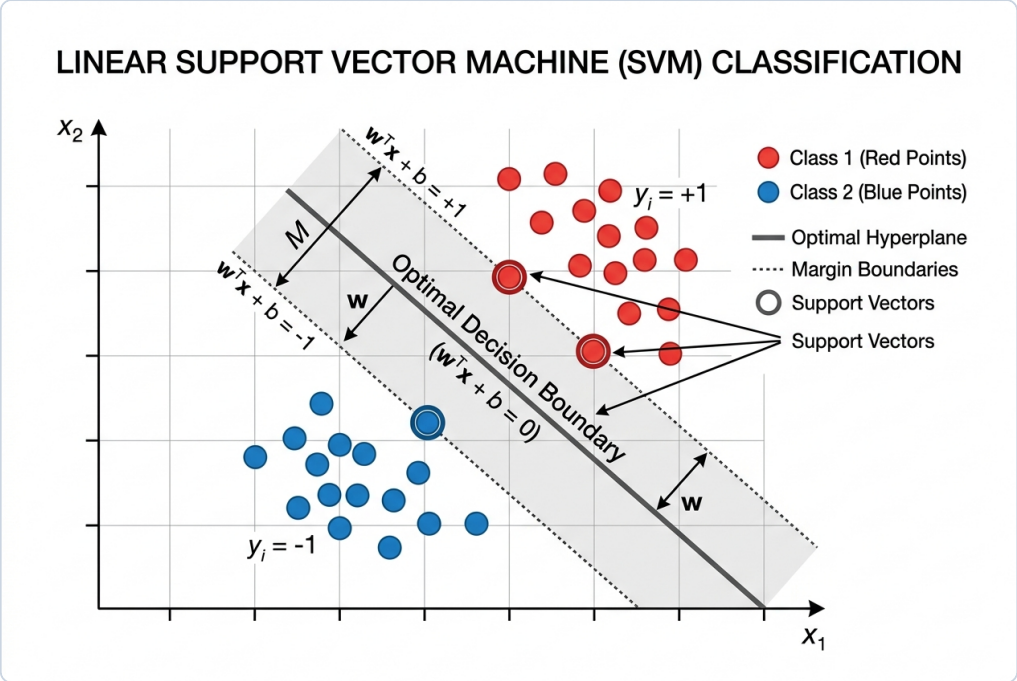


Figure 6.1: SVM classification, showing margin boundaries and support vectors.

Example Implementation

```
from sklearn.svm import SVC
from sklearn.naive_bayes import GaussianNB
import numpy as np

# Sample SVM classification
X = np.array([[1, 2], [2, 3], [3, 1], [6, 5], [7, 8], [8, 6]])
y = np.array([0, 0, 0, 1, 1, 1])

clf = SVC(kernel='linear').fit(X, y)
print("Support Vectors:
", clf.support_vectors_)
```

Exercises

EASY

Exercise 1: Naive Bayes Text Classification

Train a `MultinomialNB` model with additive smoothing parameter `alpha=1.0` on the provided training set `X_train` (text TF-IDF features) and target labels `y_train`. Store the fitted model in `nb_model`.

How to solve: Import `MultinomialNB` from `sklearn.naive_bayes`. Instantiate it with default parameters, and fit it on the TF-IDF representation of the text: `nb_model.fit(X_train, y_train)`.

TEMPLATE CODE:

```
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = [
    'I love learning machine learning',
    'Spam offer click here to win money',
    'Python programming is awesome',
    'Urgent lottery cash prize claim now'
]
labels = [0, 1, 0, 1] # 0 = ham, 1 = spam
vectorizer = TfidfVectorizer()
X_train = vectorizer.fit_transform(corpus)
y_train = np.array(labels)

# TODO: Train nb_model
nb_model = _____
```

MEDIUM

Exercise 2: SVM Support Vectors

Train a Support Vector Classifier (`SVC`) on the synthetic 2D data below using a `'linear'` kernel and parameter `C=1.0` (set `random_state=42`). Store your trained model in `svm_model`. Extract the indices of the support vectors from the model and store them in an array called `support_indices`.

How to solve: Train `SVC(kernel='linear')` on the dataset. The indices of the support vectors are stored in the `support_` attribute of the fitted model.

TEMPLATE CODE:

```
from sklearn.svm import SVC
from sklearn.datasets import make_blobs
X_train, y_train = make_blobs(n_samples=100, centers=2, random_state=42, clu
ster_std=1.2)

# TODO: Fit svm_model and extract support_indices
svm_model = ____
support_indices = ____

print("Support vector indices:", support_indices)
```

HARD

Exercise 3: Linear vs. RBF Kernels on Non-linear Data

Compare the effectiveness of a Linear Kernel vs. RBF Kernel on a non-linearly separable moons dataset. 1. Fit a linear SVM (`SVC(kernel='linear', random_state=42)`) on the training data. 2. Fit an RBF SVM (`SVC(kernel='rbf', random_state=42)`) on the training data. 3. Compute the accuracy of both models on the test set. Store your results in variables named `acc_linear` and `acc_rbf`.

How to solve: Train an SVM with a linear kernel: `SVC(kernel='linear')`, and another with an RBF kernel: `SVC(kernel='rbf')`. Compare their accuracy scores on the non-linear test dataset.

TEMPLATE CODE:

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X_moons, y_moons = make_moons(n_samples=500, noise=0.2, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X_moons, y_moons, test_s
ize=0.3, random_state=42)

# TODO: Fit both models and compute test set accuracies
acc_linear = ____
acc_rbf = ____

print(f"Linear SVM Accuracy: {acc_linear:.4f}")
print(f"RBF SVM Accuracy: {acc_rbf:.4f}")
```

CHAPTER 7

Unsupervised Learning & Clustering

Theory & Foundations

Unsupervised learning operates on unlabeled data to find hidden groupings or reduce dimensions. In this notebook, we cover: 1. **Principal Component Analysis (PCA)**: Linear dimensionality reduction. It projects data onto orthogonal axes that maximize variance. 2. **K-Means Clustering**: Partitions data into k distinct clusters by minimizing the sum of squared distances (inertia) between points and centroids. 3. **Elbow Method**: A technique to identify the optimal number of clusters k by plotting inertia vs. k.

Deep Dive: Dimensional Projection & Centroid Convergence

Principal Component Analysis (PCA) is an unsupervised technique that finds orthogonal axes (Principal Components) along which the variance of the data is maximized. It projects high-dimensional data into a lower-dimensional space while retaining as much informational variance as possible. **K-Means** is a clustering algorithm that partitions data into k groups. It iteratively: 1. Assigns each point to the nearest centroid. 2. Updates centroids to the mean of assigned points. The algorithm converges when assignments stop changing. The **Elbow Method** evaluates clustering quality by plotting the sum of squared distances (inertia) against different values of k. The "elbow" point represents the optimal trade-off between clustering detail and simplicity.

Example Implementation

```
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

# Load data
iris = load_iris()
X = iris.data

# Apply PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
print("Original shape:", X.shape)
print("PCA shape:", X_pca.shape)
```

Exercises

EASY

Exercise 1: Dimension Reduction with PCA

Perform PCA on the Iris dataset `X` below. 1. Reduce the dataset to **2** principal components (set `random_state` to `42`). 2. Store the projected coordinates array in a variable named `X_pca`.

How to solve: Import `PCA` from `sklearn.decomposition`. Fit it and transform the data using `pca.fit_transform(X)`, setting `n_components=2`.

TEMPLATE CODE:

```
from sklearn.datasets import load_iris
iris = load_iris()
X = iris.data

# TODO: Fit PCA and transform X to 2 dimensions
X_pca = ____

print("X_pca shape:", X_pca.shape)
```

MEDIUM

Exercise 2: K-Means & The Elbow Curve

Find the optimal number of clusters for the PCA-reduced Iris dataset `X_pca` using the Elbow Method. Write a loop that fits `KMeans` models for k values from **1 to 5** (inclusive). For each model: 1. Use `random_state=42`. 2. Extract the model's `inertia_` (reconstruction error). 3. Append this inertia to a list called `inertias`.

How to solve: Loop over cluster counts k from 1 to 10. Fit `KMeans` for each k, extract the inertia using `kmeans.inertia_`, and plot these values to find the elbow point.

TEMPLATE CODE:

```
from sklearn.cluster import KMeans

# TODO: Run K-Means for k=1..5 and store inertias
inertias = []

print("Inertias list:", inertias)
```

HARD

Exercise 3: K-Means Clustering from Scratch

Implement the K-Means clustering algorithm from scratch in NumPy. Write a function `kmeans_scratch(X, k, max_iters=100)` that: 1. Initializes centroids by randomly selecting k unique rows from X. Use `np.random.seed(42)` and `np.random.choice` for consistency. 2. Loops up to `max_iters`. In each iteration: - Assigns each data point to the nearest centroid (use Euclidean distance). - Recalculates centroids as the mean of all points assigned to them. - If centroids do not change, breaks early. 3. Returns `(centroids, labels)` as numpy arrays.

How to solve: In the loop: 1. Calculate distances from each point to each centroid. 2. Assign points to the closest centroid using `np.argmin(distances, axis=1)`. 3. Update centroids by computing the mean of points in each cluster: `np.mean(X[assignments == i], axis=0)`.

TEMPLATE CODE:

```
def kmeans_scratch(X, k, max_iters=100):
    # TODO: Implement K-Means from scratch
    pass

# Test our function
np.random.seed(42)
X_test = np.concatenate([
    np.random.normal(loc=[0, 0], scale=0.5, size=(50, 2)),
    np.random.normal(loc=[5, 5], scale=0.5, size=(50, 2))
])
centers, labels = kmeans_scratch(X_test, 2)
print("Centers:
", centers)
```

CHAPTER 8

Feature Engineering & Selection

Theory & Foundations

Feature engineering is the process of using domain knowledge to create new features that help machine learning algorithms perform better. Feature selection helps discard redundant or noisy features. In this notebook, we cover: 1. **Polynomial Features**: Capture non-linear interactions. 2. **Cyclical Features**: Handle cyclic temporal data (hours, months) using sine/cosine transforms. 3. **Recursive Feature Elimination (RFE)**: Fit model iteratively, removing the least important features.

Deep Dive: Feature Space Expansion and Selection

Machine learning models perform best when features match the underlying patterns. **Polynomial features** expand the feature space by creating multiplicative interactions (e.g., x_1^2, $x_1 x_2$), allowing linear models to fit non-linear curves. **Cyclical feature encoding** maps periodic variables (e.g., hour, month) using trigonometric functions:

$$x_{sin} = \sin((2 \pi x)/(period)), x_{cos} = \cos((2 \pi x)/(period))$$

This preserves the temporal proximity of end and start boundaries (e.g., 23:00 and 00:00). **Recursive Feature Elimination (RFE)** is a wrapper-based feature selection method. It trains a model, ranks features by importance or coefficients, prunes the least important feature, and repeats until the target number of features is reached.

Example Implementation

```
from sklearn.preprocessing import PolynomialFeatures
import pandas as pd
import numpy as np

# Example: Polynomial features
X = np.array([[2, 3]])
poly = PolynomialFeatures(degree=2, include_bias=False)
print("Poly Features of [2, 3]:", poly.fit_transform(X))
```

Exercises

EASY

Exercise 1: Polynomial and Interaction Features

Create polynomial features of degree **2** (excluding the bias term) for the provided features array `X`. Store the output in `X_poly`.

How to solve: Import `PolynomialFeatures` from `sklearn.preprocessing`. Fit and transform features with `degree=2` and `include_bias=False` to generate interaction and polynomial columns.

TEMPLATE CODE:

```
X = np.array([[2.0, 3.0], [4.0, 5.0]])

# TODO: Fit PolynomialFeatures (degree=2, include_bias=False) and transform X
X_poly = ____

print("X_poly:
", X_poly)
```

MEDIUM

Exercise 2: Cyclical Time Feature Extraction

Time columns like hours (0-23) have a circular structure (hour 23 is close to hour 0). Given a DataFrame `df` with a datetime column `'timestamp'`: Extract the hour and convert it into **sine and cosine components**. Create two new columns `'hour_sin'` and `'hour_cos'`. **Formula**:

$$\text{hour}_{\text{sin}} = \sin((2\ \pi \cdot \text{hour})/(24))$$

$$\text{hour}_{\text{cos}} = \cos((2\ \pi \cdot \text{hour})/(24))$$

Store the updated DataFrame in `df_processed`.

How to solve: Map the cyclical hour values to sine and cosine features: `sin_hour = np.sin(2 * np.pi * df['Hour'] / 24)` and `cos_hour = np.cos(2 * np.pi * df['Hour'] / 24)`.

TEMPLATE CODE:

```
df = pd.DataFrame({
    'timestamp': pd.date_range('2026-06-27 00:00:00', periods=4, freq='6h')
})

# TODO: Extract hour and compute sine/cosine encodings
df_processed = df.copy()
# df_processed['hour_sin'] = ...
# df_processed['hour_cos'] = ...

print(df_processed)
```

HARD

Exercise 3: Recursive Feature Elimination

Use Recursive Feature Elimination (`RFE`) with a `RandomForestClassifier(random_state=42)` estimator to select the top **3** features of the dataset `X`. Store: 1. The fitted RFE model object in `rfe_model`. 2. The selected features subset of X in `X_selected`. 3. The RFE features ranking array (rank 1 indicates selected features) in `ranking`.

How to solve: Import `RFE` from `sklearn.feature_selection`. Pass a base estimator (like Logistic Regression) and specify the number of features to select, then call `.fit(X, y)`.

TEMPLATE CODE:

```
from sklearn.feature_selection import RFE
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=500, n_features=8, n_informative=3, rand
om_state=42)

# TODO: Perform RFE to select top 3 features using RandomForestClassifier
rfe_model = ____
X_selected = ____
ranking = ____

print("Feature Ranking:", ranking)
print("Selected Features Shape:", X_selected.shape)
```

CHAPTER 9

Model Evaluation & Hyperparameter Tuning

Theory & Foundations

Evaluating and tuning models is crucial to verify they generalize to unseen data. In this notebook, we cover: 1. **Cross-Validation**: K-Fold cross validation splits the dataset into K parts, using $K-1$ for training and 1 for validation in rotation. 2. **Hyperparameter Tuning**: Searching the parameter space using Grid Search (brute force) or Random Search (stochastic). 3. **Classification Threshold Tuning**: Finding the optimal classification probability cutoff to maximize metrics like the F1-score.

Deep Dive: Model Validation and Evaluation Diagnostics

Evaluating models on training data leads to optimistic bias. **K-Fold Cross-Validation** splits data into k non-overlapping folds. The model is trained on $k-1$ folds and validated on the remaining fold, repeating k times. This provides a robust distribution of performance. For hyperparameter tuning, **Grid Search** searches every combination in a specified grid, guaranteeing the best parameters are found at high computational cost. **Random Search** samples hyperparameter combinations randomly, covering a wider range in less time. In classification, standard accuracy fails on imbalanced data. We evaluate **F1-Score** (harmonic mean of precision and recall) and adjust the classification probability threshold to optimize the model for specific business requirements.

Example Implementation

```
from sklearn.model_selection import KFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=100, random_state=42)
rf = RandomForestClassifier(n_estimators=10, random_state=42)
scores = cross_val_score(rf, X, y, cv=5)
print("CV Scores:", scores)
print("Mean CV Score:", scores.mean())
```

Exercises

EASY

Exercise 1: K-Fold Cross-Validation

Perform a 5-split K-Fold Cross-Validation (`KFold` with `n_splits=5`, `shuffle=True`, and `random_state=42`) using `cross_val_score` on the provided `RandomForestClassifier` (`rf`) and datasets. Compute and store the mean of the cross-validation scores in `mean_cv_score`.

How to solve: Import `cross_val_score` from `sklearn.model_selection`. Pass the classifier, features, labels, and set the number of folds (e.g. `cv=5`).

TEMPLATE CODE:

```
from sklearn.model_selection import KFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=1000, n_features=10, random_state=42)
rf = RandomForestClassifier(n_estimators=50, max_depth=5, random_state=42)

# TODO: Configure KFold and compute mean_cv_score
mean_cv_score = ____

print("Mean CV Accuracy Score:", mean_cv_score)
```

MEDIUM

Exercise 2: Grid Search vs. Randomized Search Tuning

Tune the parameters of a `DecisionTreeRegressor` using Grid Search and Random Search. Search space: - `'max_depth'`: `[3, 5, 7]` - `'min_samples_split'`: `[2, 5, 10]`

1. Fit `GridSearchCV` on the dataset `X, y`.
2. Fit `RandomizedSearchCV` on the dataset `X, y` with `n_iter=5` and `random_state=42`. Store the dictionaries of the best parameters found in variables named `best_params_grid` and `best_params_random` respectively.

How to solve: Use `GridSearchCV` to search all parameter combinations, and `RandomizedSearchCV` to search a random subset of parameters. Measure and compare execution time and accuracy.

TEMPLATE CODE:

```
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
from sklearn.tree import DecisionTreeRegressor
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=500, n_features=5, noise=0.1, random_state=4
2)

# TODO: Fit Grid Search and Random Search. Extract best parameters.
best_params_grid = ____
best_params_random = ____

print("Grid Search Best Parameters:", best_params_grid)
print("Random Search Best Parameters:", best_params_random)
```

HARD

Exercise 3: F1-Score Classification Threshold Tuning

By default, classifiers predict class 1 if the probability is ≥ 0.5. In imbalanced sets, this threshold might not be optimal. Given predictions probabilities `y_probs` and true targets `y_test`: Find the classification threshold in the range $[0.1, 0.9]$ (inclusive, with steps of `0.01`) that maximizes the F_1 score. Store: 1. The optimal threshold value in `best_threshold`. 2. The maximum F1 score in `max_f1`.

How to solve: Use `model.predict_proba(X_test)[:, 1]` to get probabilities. Loop through thresholds, convert probabilities to binary predictions, and calculate the F1-score to find the best threshold.

TEMPLATE CODE:

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=500, n_classes=2, weights=[0.7, 0.3], ra
ndom_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, ran
dom_state=42)
clf = LogisticRegression(random_state=42).fit(X_train, y_train)
y_probs = clf.predict_proba(X_test)[:, 1]

# TODO: Find threshold in np.arange(0.1, 0.91, 0.01) maximizing F1-score
best_threshold = 0.0
max_f1 = 0.0

print(f"Optimal Threshold: {best_threshold:.2f} with Max F1: {max_f1:.4f}")
```

CHAPTER 10

Introduction to Deep Learning

Theory & Foundations

Deep Learning builds models using Artificial Neural Networks (ANN). In this notebook, we cover: 1. **Perceptron**: The simplest neural network block, modeling a single neuron:

$$y = \text{activation}(\mathbf{w} \cdot \mathbf{x} + b)$$

1. **Multi-Layer Perceptron (MLP)**: A feedforward network with one or more hidden layers.
2. **Activation Functions**: Introduce non-linearities. Common ones are ReLU, Sigmoid, and Tanh.
3. **Backpropagation**: Adjusts weights in the network using gradients computed via the chain rule.

Deep Dive: Deep Neural Architecture & Backpropagation

A **Perceptron** is the basic unit of a neural network. It computes a weighted sum of inputs, adds a bias, and passes the output through a step function. A single perceptron can only learn linear boundaries. To learn non-linear patterns, **Multi-Layer Perceptrons (MLPs)** stack layers of neurons. Hidden layers use non-linear activation functions (like Sigmoid or ReLU) to project data into non-linear spaces. Neural networks learn via **Backpropagation**: 1. **Feedforward**: Input passes through layers to generate predictions. 2. **Loss Calculation**: Computes error compared to targets. 3. **Backpropagation**: The error gradient is propagated backward through the network using the calculus chain rule. 4. **Weight Updates**: Weights are updated using Gradient Descent to minimize the loss.

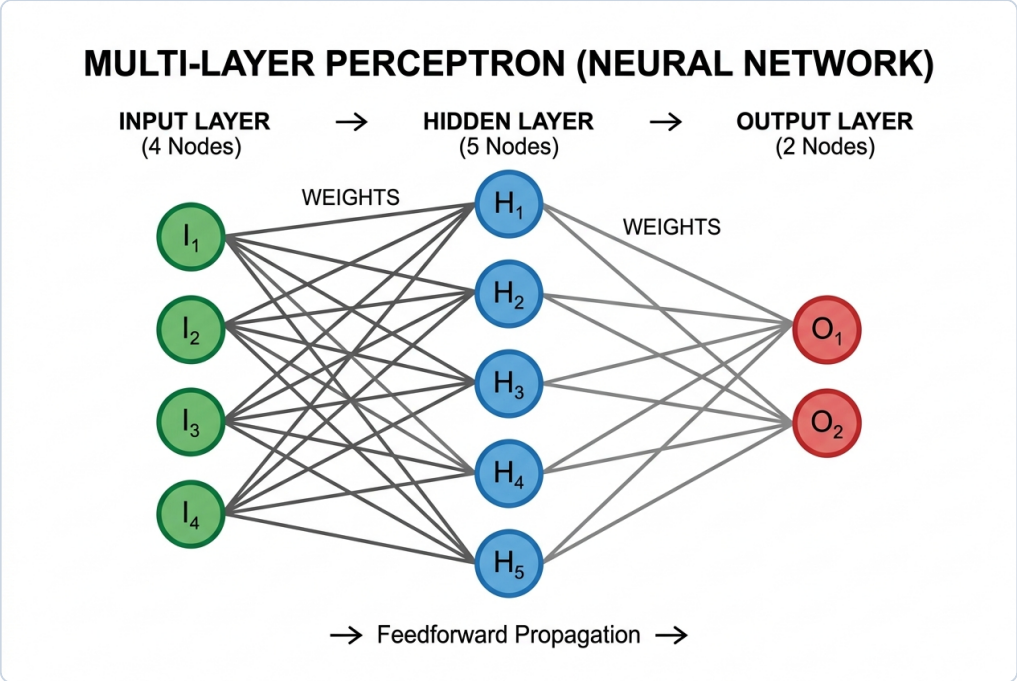


Figure 10.1: Architecture of a Multi-Layer Perceptron (feedforward neural network).

Example Implementation

```
from sklearn.neural_network import MLPClassifier
import numpy as np

# Simple MLP model
X = np.array([[0.0, 0.0], [1.0, 1.0]])
y = np.array([0, 1])

mlp = MLPClassifier(hidden_layer_sizes=(4,), max_iter=1000, random_state=42)
mlp.fit(X, y)
print("Predictions:", mlp.predict(X))
```

Exercises

EASY

Exercise 1: Multi-Layer Perceptron Classifier

Train a scikit-learn `MLPClassifier` on features `X_train` and labels `y_train` below. Configure the model with: 1. Two hidden layers of size `64` and `32` respectively (i.e. `(64, 32)`). 2. `'relu'` activation function. 3. `random_state=42`.

Store the fitted model in `mlp_model`.

How to solve: Import `MLPClassifier` from `sklearn.neural_network`. Train it with `hidden_layer_sizes=(8, 8)`, `activation='relu'`, and `random_state=42`.

TEMPLATE CODE:

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=800, n_features=8, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
dom_state=42)

# TODO: Fit mlp_model
mlp_model = ____
```

MEDIUM

Exercise 2: Perceptron from Scratch

Implement a single-neuron Perceptron model to solve the logical **AND** gate. Write two functions: 1. `perceptron_predict(x, weights, bias)`: returns `1` if $\mathbf{w} \cdot \mathbf{x} + b \geq 0$ else `0`. 2. `perceptron_train(X, y, lr, epochs)`: trains the weights and bias using the Perceptron update rule. Initialize weights to zeros and bias to `0.0`. - Update rule: if prediction $\hat{y}_i \neq y_i$, update:

$$\mathbf{w} \leftarrow \mathbf{w} + lr \cdot (y_i - \hat{y}_i) \cdot \mathbf{x}_i$$

$$b \leftarrow b + lr \cdot (y_i - \hat{y}_i)$$

- Returns learned `(weights, bias)`.

How to solve: Loop over samples. If prediction matches target, do nothing. If they mismatch, update weights: `w += lr * (target - pred) * x`, and update bias: `b += lr * (target - pred)`.

TEMPLATE CODE:

```
def perceptron_predict(x, weights, bias):
    # TODO: Implement inference
    pass

def perceptron_train(X, y, lr=0.1, epochs=100):
    # TODO: Implement training loop
    weights = np.zeros(X.shape[1])
    bias = 0.0
    return weights, bias

# Test perceptron training on AND gate
X_and = np.array([[0,0], [0,1], [1,0], [1,1]])
y_and = np.array([0, 0, 0, 1])
w_and, b_and = perceptron_train(X_and, y_and)
print("Learned Weights:", w_and)
print("Learned Bias:", b_and)
```

HARD

Exercise 3: Two-Layer Neural Network from Scratch

Build a 2-layer Neural Network (Input → Hidden → Output) from scratch using NumPy to solve the non-linear **XOR** gate. - Hidden activation: **Sigmoid** - Output activation: **Sigmoid**

Write a class `SimpleNN` containing: 1. `__init__(self, input_dim, hidden_dim, output_dim)`: Initializes random weights and zero biases. 2. `forward(self, X)`: Computes and returns predictions. Stores internal activations for backpropagation. 3. `backward(self, X, y, lr)`: Computes derivatives and updates weights and biases using gradient descent. 4. `train(self, X, y, lr, epochs)`: Performs training loop.

How to solve: Implement feedforward: `a1 = sigmoid(np.dot(X, W1) + b1), a2 = sigmoid(np.dot(a1, W2) + b2)`. Backpropagate errors by calculating gradients using derivative of sigmoid: `a * (1 - a)`.

TEMPLATE CODE:

```
# TODO: Implement the SimpleNN class
class SimpleNN:
    def __init__(self, input_dim, hidden_dim, output_dim):
        pass

    def forward(self, X):
        pass

    def backward(self, X, y, lr):
        pass

    def train(self, X, y, lr=0.1, epochs=10000):
        pass

# Test XOR gate
X_xor = np.array([[0,0], [0,1], [1,0], [1,1]])
y_xor = np.array([[0], [1], [1], [0]])

nn = SimpleNN(input_dim=2, hidden_dim=4, output_dim=1)
# nn.train(X_xor, y_xor, lr=0.1, epochs=10000)
# print("Predictions after training:
", nn.forward(X_xor))
```

CHAPTER 11

Time Series Analysis & Forecasting

Theory & Foundations

Time series data is a sequence of data points indexed in chronological order. Unlike standard datasets, observations in a time series are usually sequentially dependent.

1. Components of Time Series

A time series Y_t is typically decomposed into: - **Trend (T_t)**: The long-term increase or decrease in the data. - **Seasonality (S_t)**: Patterns that repeat at regular intervals (e.g., daily, weekly, monthly). - **Cyclical (C_t)**: Fluctuations that are not of a fixed period (usually economic cycles). - **Irregular/Noise (I_t)**: Random, unpredictable residual fluctuations.

2. Stationarity

A time series is **stationary** if its statistical properties do not change over time. Specifically: 1. **Constant Mean**: $E[Y_t] = \mu$ for all t. 2. **Constant Variance**: $Var(Y_t) = \sigma^2$ for all t. 3. **Constant Autocovariance**: $Cov(Y_t, Y_{t-k})$ depends only on lag k, not on time t.

Most forecasting models (like ARIMA) assume stationarity. We test stationarity using visual plots (rolling stats) or statistical tests like the **Augmented Dickey-Fuller (ADF) test**. If the series is non-stationary, we apply **differencing**:

$$Y'_t = Y_t - Y_{t-1}$$

3. ARIMA Models

An Autoregressive Integrated Moving Average model is denoted as $ARIMA(p, d, q)$: - **AR (p)**: Autoregressive part uses dependency between an observation and lagged observations. - **I (d)**: Differencing degree required to make the series stationary. - **MA (q)**: Moving Average part models dependency between an observation and residual errors from lagged forecasts.

Deep Dive: Time-Series Dynamics and Autoregressive Fitting

Time series data contains sequential observations where order matters. Unlike standard datasets, time-series observations are often correlated with past values (**autocorrelation**). A time series is

stationary if its statistical properties (mean, variance, covariance) are constant over time. Non-stationary data is hard to model. **Differencing** ($y'_t = y_t - y_{t-1}$) removes trends and seasonality to achieve stationarity, which is verified using the statistical **Augmented Dickey-Fuller (ADF) test**. **ARIMA** models stationary time series by combining: - **Autoregression (AR)**: Predicting current values based on past values. - **Integration (I)**: The degree of differencing needed. - **Moving Average (MA)**: Modeling relationship with past forecast errors.

Example Implementation

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA

# Generate random walk with trend
np.random.seed(42)
time = pd.date_range(start='2026-01-01', periods=100, freq='D')
value = np.sin(np.linspace(0, 20, 100)) + np.random.normal(scale=0.1, size=100) + n
p.linspace(0, 5, 100)
df = pd.DataFrame({'value': value}, index=time)

# Visualizing Rolling Statistics
rolling_mean = df['value'].rolling(window=7).mean()
rolling_std = df['value'].rolling(window=7).std()

plt.figure(figsize=(10, 5))
plt.plot(df['value'], label='Original')
plt.plot(rolling_mean, label='Rolling Mean (7d)', color='red')
plt.plot(rolling_std, label='Rolling Std (7d)', color='black')
plt.legend()
plt.title("Rolling Statistics Visualization")
plt.show()

# Perform ADF Test
result = adfuller(df['value'])
print(f"ADF Statistic: {result[0]:.4f}")
print(f"p-value: {result[1]:.4f}")
```

Exercises

EASY

Exercise 1: Rolling Statistics

Given the DataFrame `df` containing a daily time series: 1. Calculate the **7-day rolling mean** of `'value'` and store it in a Series named `rolling_mean`. 2. Calculate the **7-day rolling standard deviation** of `'value'` and store it in a Series named `rolling_std`. *Note*: Keep the default settings (i.e. do not drop NaNs or change minimum periods).

How to solve: Call `.rolling(window=12)` on the series. Use `.mean()` to get rolling means, and `.std()` to get rolling standard deviations.

TEMPLATE CODE:

```
# Define time series
np.random.seed(42)
time = pd.date_range(start='2026-01-01', periods=100, freq='D')
value = np.sin(np.linspace(0, 20, 100)) + np.random.normal(scale=0.1, size=10
0)
df = pd.DataFrame({'value': value}, index=time)

# TODO: Compute rolling statistics
rolling_mean = ____
rolling_std = ____

print("Mean head:
", rolling_mean.head(10))
print("Std head:
", rolling_std.head(10))
```

MEDIUM

Exercise 2: Time Series Differencing & Stationarity Test

The time series below has a strong linear trend, making it non-stationary. 1. Perform **first-order differencing** on the `'value'` column to remove the trend. Store the resulting differenced Series (with `NaN` dropped using `.dropna()`) in `diff_series`. 2. Run the **Augmented Dickey-Fuller (ADF) test** on your `diff_series` and extract the **p-value** of the test. Store this value in `adf_pvalue`.

How to solve: Subtract the shifted series to difference: `diff_series = series.diff().dropna()`. Run the ADF test using `adfuller(diff_series)` and inspect the p-value.

TEMPLATE CODE:

```
from statsmodels.tsa.stattools import adfuller

np.random.seed(42)
time = pd.date_range(start='2026-01-01', periods=100, freq='D')
value = np.sin(np.linspace(0, 20, 100)) + np.random.normal(scale=0.1, size=10
0)
# Add linear trend
value = value + np.linspace(0, 5, 100)
df = pd.DataFrame({'value': value}, index=time)

# TODO: Difference the series once, drop NaN, and calculate ADF test p-value.
diff_series = ____
adf_pvalue = ____

print(f"ADF test p-value after differencing: {adf_pvalue:.6f}")
```

HARD

Exercise 3: Time Series Forecasting and MAPE

Fit an $\text{ARIMA}(1, 1, 0)$ model on the training time series `train`. 1. Train the model using the training data `train`. 2. Forecast the next **20 steps** into the future (matching the length of the test set). 3. Compute the **Mean Absolute Percentage Error (MAPE)** of the predictions against the actual test set `test`. Store the MAPE score in a variable named `mape`.

MAPE Formula:

$$\text{MAPE} = (100\%)/(N) \sum_{t=1}^{N} | (y_t - \hat{y}_t)/(y_t) |$$

How to solve: Fit the `ARIMA(series, order=(1,1,1))` model. Generate forecasts, and calculate MAPE: `np.mean(np.abs((y_true - y_pred) / y_true)) * 100`.

TEMPLATE CODE:

```
from statsmodels.tsa.arima.model import ARIMA

np.random.seed(42)
time = pd.date_range(start='2026-01-01', periods=100, freq='D')
value = np.sin(np.linspace(0, 20, 100)) + np.random.normal(scale=0.1, size=10
0) + np.linspace(0, 5, 100)
df = pd.DataFrame({'value': value}, index=time)

train, test = df.iloc[:80], df.iloc[80:]

# TODO: Fit ARIMA(1,1,0), forecast 20 steps, and calculate MAPE percentage
mape = ____

print(f"Forecast MAPE: {mape:.4f}%")
```

CHAPTER 12

Natural Language Processing (NLP)

Theory & Foundations

Natural Language Processing focuses on the interaction between computers and human languages. Computers require numerical matrices, so converting raw text into numbers is key.

1. Text Preprocessing

Before vectorization, we clean and standardize text: - **Lowercasing**: Converting all text to lowercase. - **Punctuation & Number Removal**: Removing symbols that do not carry semantic meaning. - **Stopword Removal**: Eliminating common words (e.g. 'the', 'is', 'and') that add noise. - **Tokenization**: Splitting sentences into individual words or tokens. - **Stemming/Lemmatization**: Reducing words to their root form (e.g., 'running' → 'run').

2. Bag-of-Words vs. TF-IDF Vectorization

- **Bag-of-Words (BoW)**: Counts occurrence of each word in a document. Ignores grammar and word order.
- **TF-IDF (Term Frequency-Inverse Document Frequency)**: Scales word count by how unique it is across all documents:

$$\text{TF}(t, d) = (\text{Count of word t in document d})/(\text{Total words in d})$$

$$\text{IDF}(t) = \log((\text{Total documents N})/(\text{Documents containing word t}))$$

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

3. Word Embeddings

Represent words as dense vectors in a continuous space, capturing semantic similarities (e.g. Word2Vec, GloVe, FastText). Words with similar meanings end up close to each other.

Deep Dive: Text Vectorization and Semantic Embeddings

Natural Language Processing (NLP) translates unstructured text into numeric matrices: - **Bag of Words (BoW)**: Counts word frequencies, ignoring syntax and word order. - **TF-IDF**: Weighs word frequency against how common the word is across the corpus:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \log((N)/(\text{DF}(t)))$$

This highlights unique, informative words. - **Cosine Similarity**: Measures the angle between document vectors to assess similarity:

$$\text{similarity} = \cos(\backslash\text{theta}) = (\mathbf{A} \cdot \mathbf{B})/(||\mathbf{A}||\ ||\mathbf{B}||)$$

- **Word Embeddings**: Map words to dense vectors in a continuous vector space where words with similar meanings are close together, capturing semantic relationships.

Example Implementation

```
import re
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

# TF-IDF Demonstration
docs = ["I love machine learning.", "Machine learning is powerful."]
vec = TfidfVectorizer()
tfidf = vec.fit_transform(docs)

# Output representation
print("Vocabulary:", vec.vocabulary_)
print("TF-IDF Matrix Shape:", tfidf.shape)

# Cosine Similarity
sim = cosine_similarity(tfidf[0], tfidf[1])
print(f"Similarity between document 0 and 1: {sim[0][0]:.4f}")
```

Exercises

EASY

Exercise 1: Basic Text Preprocessing & Token Count

Perform the following cleaning steps on the string `text` below: 1. Convert the entire text to **lowercase**. 2. Remove any **punctuation** (keep only letters and whitespaces). 3. **Tokenize** (split by whitespace) the cleaned text. 4. Remove any **stopwords** from this list: `stopwords = {'is', 'a', 'and', 'to', 'for', 'the'}`. Store the cleaned tokens list in `cleaned_tokens`. 5. Create a word frequency distribution dictionary (or Counter) named `freq_dist` counting the occurrences of each remaining token.

How to solve: Convert text to lowercase, strip punctuation using a regular expression (e.g. `re.sub(r'[^a-z0-9\s]', '', text)`), split into tokens, and filter out tokens present in the stopword list.

TEMPLATE CODE:

```
import re
from collections import Counter

text =
"Welcome to the learning of Python and NLP! NLP is a powerful tool for
learning Python."
stopwords = {'is', 'a', 'and', 'to', 'for', 'the'}

# TODO: Preprocess the text and calculate word counts
cleaned_tokens = ____
freq_dist = ____

print("Cleaned Tokens:", cleaned_tokens)
print("Frequency Distribution:", freq_dist)
```

MEDIUM

Exercise 2: TF-IDF Document Cosine Similarity

Calculate the **Cosine Similarity Matrix** between the TF-IDF representation of three documents. 1. Use `TfidfVectorizer` (with default parameters) to construct TF-IDF representations of the documents list `docs`. 2. Compute the cosine similarity between each pair of documents. Store the resulting $(3,3)$ numpy matrix in `similarity_matrix`.

How to solve: Fit `TfidfVectorizer` on the preprocessed corpus. Compute the similarity matrix by calling `cosine_similarity(tfidf_matrix)`.

TEMPLATE CODE:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

docs = [
    "cats eat fish and love milk",
    "dogs chase cats and love bones",
    "fish swim in the deep water"
]

# TODO: Fit TF-IDF and calculate the cosine similarity matrix
similarity_matrix = ____

print("Similarity Matrix:
", similarity_matrix)
```

HARD

Exercise 3: Word Embedding Sentiment Classifier

Given pre-calculated 3D word vectors `word_vectors` and a dataset of sentences: 1. Represent each sentence by **averaging** the vectors of its constituent words. (If a word is not in the dictionary, ignore it). Store the resulting $(4, 3)$ numpy array in `sentence_vectors`. 2. Train a `LogisticRegression` model on the first **2** sentences (`sentence_vectors[:2]`) using labels `[1, 0]`. 3. Predict labels of the remaining **2** test sentences (`sentence_vectors[2:]`) and evaluate model accuracy against targets `[1, 0]`. Store the test accuracy score in `test_acc`.

> **How to solve:** For each document, average the embedding vectors of all words in the document. Train a `LogisticRegression` model on these document vectors.

TEMPLATE CODE:

```
import numpy as np
from sklearn.linear_model import LogisticRegression

word_vectors = {
    'happy': np.array([0.5, 0.8, -0.1]),
    'sad': np.array([-0.6, -0.5, 0.2]),
    'good': np.array([0.4, 0.6, 0.0]),
    'bad': np.array([-0.5, -0.7, 0.1]),
    'day': np.array([0.1, 0.1, 0.1]),
    'night': np.array([-0.1, -0.1, 0.0])
}

sentences = [
    "happy good day",    # Positive
    "sad bad night",     # Negative
    "good day",          # Positive (Test)
    "bad night"          # Negative (Test)
]
labels = [1, 0, 1, 0] # 1 = Positive, 0 = Negative

# TODO: Compute sentence embeddings by averaging word vectors, fit Logistic
Regression, and compute test_acc
sentence_vectors = ____
test_acc = ____

print("Sentence Vectors:
", sentence_vectors)
print(f"Test Accuracy: {test_acc:.2f}")
```

CHAPTER 13

Anomaly Detection

Theory & Foundations

Anomaly detection is the identification of rare items, events, or observations which raise suspicions by differing significantly from the majority of the data.

1. Statistical Outlier Detection

- **Z-Score**: Measures how many standard deviations a data point is from the mean. Outliers are typically identified if:

$$|Z| > 3 \text{ where } Z = (x - \mu)/(\sigma)$$

- **Interquartile Range (IQR)**: Calculates the spread of the middle 50% of the data. Outliers lie outside the bounds:

$$\text{Lower Bound} = Q_1 - 1.5 \times \text{IQR}$$

$$\text{Upper Bound} = Q_3 + 1.5 \times \text{IQR}$$

Where $\text{IQR} = Q_3 - Q_1$, and Q_1, Q_3 are the 25th and 75th percentiles.

2. Isolation Forest

An unsupervised tree-based algorithm. Anomalies are isolated closer to the root of the trees because they require fewer splits to partition from the rest of the dataset.

3. Mahalanobis Distance

For multidimensional data, Mahalanobis distance accounts for the correlation between features:

$$D_M(\mathbf{x}) = \sqrt{((\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}) (\mathbf{x} - \boldsymbol{\mu})}$$

Where $\boldsymbol{\mu}$ is the mean vector and $\boldsymbol{\Sigma}$ is the covariance matrix.

Deep Dive: Outlier Boundaries and Distance Metrics

Anomalies are rare observations that deviate significantly from standard distributions: - **Statistical Thresholds**: Z-score marks points beyond a standard deviation limit ($|Z| > 3$). IQR defines outliers beyond $[Q1 - 1.5IQR, Q3 + 1.5IQR]$. - **Isolation Forest**: An unsupervised algorithm that isolates anomalies by partitioning features. Since anomalies have extreme values, they require fewer splits to isolate, resulting in shorter paths in decision trees. - **Mahalanobis Distance**: Measures distance between a point and a distribution in multi-dimensional space, accounting for correlations between variables:

$$D_M(\mathbf{x}) = \sqrt{((\mathbf{x} - \boldsymbol{\mu})^T \boldsymbol{\Sigma}^{-1}) (\mathbf{x} - \boldsymbol{\mu})}$$

where $\boldsymbol{\Sigma}^{-1}$ is the inverse covariance matrix.

Example Implementation

```
import numpy as np
from sklearn.ensemble import IsolationForest
import matplotlib.pyplot as plt

# Generate normal data
np.random.seed(42)
X = np.random.normal(loc=0.0, scale=1.0, size=(100, 2))
# Add anomaly outlier
X = np.vstack([X, [4.0, 4.0]])

# Fit Isolation Forest
clf = IsolationForest(contamination=0.01, random_state=42)
preds = clf.fit_predict(X) # 1 = normal, -1 = anomaly

print("Outlier prediction count:", np.sum(preds == -1))
```

Exercises

EASY

Exercise 1: Z-Score vs. IQR Outliers

Identify outliers in a 1D dataset using statistical metrics. 1. Compute the Z-scores of array `X`. Identify any outliers where $|Z| > 3$. Store these outliers array in `outliers_z`. 2. Compute the Q_1 (25th percentile) and Q_3 (75th percentile) of `X`. Use the IQR method to find outliers (lying outside $[Q_1 - 1.5 \cdot IQR, Q_3 + 1.5 \cdot IQR]$). Store these outliers array in `outliers_iqr`.

How to solve: Z-scores are calculated as `(x - mean) / std`. IQR is `Q3 - Q1`, and outliers are points outside the range `[Q1 - 1.5 * IQR, Q3 + 1.5 * IQR]`.

TEMPLATE CODE:

```
np.random.seed(42)
X = np.random.normal(loc=10.0, scale=2.0, size=100)
# Inject manual anomalies
X[15] = 25.0
X[65] = -5.0

# TODO: Detect outliers using Z-score and IQR methods
outliers_z = ____
outliers_iqr = ____

print("Z-Score outliers:", outliers_z)
print("IQR outliers:", outliers_iqr)
```

MEDIUM

Exercise 2: Isolation Forest Anomaly Scoring

Train an `IsolationForest` on the 2D dataset `X` below. 1. Initialize the estimator with `contamination=0.02` and `random_state=42`. Fit it on `X`. 2. Retrieve the anomaly scores (mean anomaly score of the forest) using the `decision_function()` method. Store the scores in a numpy array named `scores`. 3. Predict the anomaly labels (1 for inliers, -1 for outliers) using the `predict()` method. Store these labels in a numpy array named `anomalies`.

How to solve: Fit `IsolationForest(contamination=0.05, random_state=42)`. Call `.predict(X)`, where anomalies are labeled as `-1`.

TEMPLATE CODE:

```
from sklearn.ensemble import IsolationForest

np.random.seed(42)
normal_data = np.random.normal(loc=0.0, scale=0.5, size=(100, 2))
anomalous_data = np.array([[3.0, 3.0], [-3.0, -3.0]])
X = np.concatenate([normal_data, anomalous_data], axis=0)

# TODO: Fit IsolationForest, extract anomaly scores and predictions
scores = ____
anomalies = ____

print("Anomaly predictions (1=normal, -1=outlier):
", anomalies[-5:])
print("Scores for outliers:", scores[-2:])
```

HARD

Exercise 3: Mahalanobis Distance from Scratch

Implement the Mahalanobis Distance calculation from scratch in NumPy. Write a function `mahalanobis_distance(X)` that: 1. Calculates the mean vector $\boldsymbol{\mu}$ of features (axis=0). 2. Computes the covariance matrix $\boldsymbol{\Sigma}$ of the dataset (Hint: use `np.cov(X.T)`). 3. Computes the inverse of the covariance matrix $\boldsymbol{\Sigma}^{-1}$ (Hint: use `np.linalg.inv`). 4. For each row $\mathbf{x}$ in X, computes the Mahalanobis distance:

$$\mathrm{d} = \sqrt{((\mathbf{x} - \boldsymbol{\mu})^{\mathrm{T}} \boldsymbol{\Sigma}^{-1}) (\mathbf{x} - \boldsymbol{\mu})}$$

1. Returns a 1D numpy array of distances for all rows in X.

How to solve: Calculate the inverse covariance matrix using `np.linalg.inv(np.cov(X.T))`. Calculate Mahalanobis distances using the formula: `d = np.sqrt(np.sum(diff @ inv_cov * diff, axis=1))`.

TEMPLATE CODE:

```
def mahalanobis_distance(X):
    # TODO: Implement Mahalanobis Distance calculation
    pass

# Test Mahalanobis distance
X_test = np.array([[1.0, 2.0], [2.0, 3.0], [5.0, 6.0]])
print("Distances:", mahalanobis_distance(X_test))
```

CHAPTER 14

Ensemble Learning & Gradient Boosting

Theory & Foundations

Ensemble learning combines predictions from multiple models to generate a single, more robust prediction.

1. Types of Ensembles

- **Voting/Averaging**: Combines predictions from different model classes (e.g. SVM + Decision Tree + Logistic Regression).
- *Hard Voting*: Class predictions are decided by majority vote.
- *Soft Voting*: Prediction is based on the argmax of the sum of predicted probabilities.
- **Bagging (Bootstrap Aggregation)**: Trains multiple models of the same class in parallel on different bootstrapped datasets (e.g., Random Forest). Reduces variance.
- **Boosting**: Trains models sequentially. Each new model focuses on correcting the errors (residuals) of the previous models. Reduces bias.
- **Stacking**: Trains base models on the training data, then uses their predictions as features to train a meta-classifier.

2. Boosting Algorithms

- **AdaBoost**: Re-weights samples at each step, putting more emphasis on previously misclassified observations.
- **Gradient Boosting**: Fits new estimators to the negative gradient (pseudo-residuals) of the loss function. Popular optimized implementations: **XGBoost**, **LightGBM**, and **CatBoost**.

Deep Dive: Ensemble Architecture & Boosting Mechanics

Ensemble learning combines multiple models to reduce error: - **Voting Classifiers**: Combine different models. **Hard voting** uses majority vote. **Soft voting** averages predicted probabilities, weighting confident predictions higher. - **Boosting**: Models are trained sequentially. **AdaBoost** assigns higher weights to samples misclassified by previous models. **Gradient Boosting** fits new models to the residual errors (gradients) of the loss function. - **Stacking**: Trains base models on the training set, uses their predictions as features, and trains a meta-model to make the final prediction.

Example Implementation

```
from sklearn.ensemble import VotingClassifier, AdaBoostClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

# Base models
clf1 = LogisticRegression(random_state=42)
clf2 = DecisionTreeClassifier(max_depth=3, random_state=42)

# Voting ensemble
voting = VotingClassifier(estimators=[('lr', clf1), ('dt', clf2)], voting='soft')
X, y = make_classification(n_samples=100, random_state=42)
voting.fit(X, y)
print("Voting Score:", voting.score(X, y))
```

Exercises

EASY

Exercise 1: Soft Voting Classifier

Create a `VotingClassifier` consisting of three base models: 1. Logistic Regression: `LogisticRegression(random_state=42)` 2. Decision Tree: `DecisionTreeClassifier(max_depth=3, random_state=42)` 3. Gaussian Naive Bayes: `GaussianNB()`

Configure the voting strategy to `'soft'`. Fit the ensemble on training datasets `X_train` and `y_train`. Store the fitted model in `voting_clf` and evaluate its accuracy score on the test set `X_test`, storing it in `test_acc`.

How to solve: Instantiate `VotingClassifier(estimators, voting='soft')`. Ensure the base estimators support probability output, fit the ensemble, and score on the test set.

TEMPLATE CODE:

```
from sklearn.ensemble import VotingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=500, n_features=6, n_classes=2, random_s
tate=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
dom_state=42)

# TODO: Fit soft voting ensemble and calculate test set accuracy
voting_clf = ____
test_acc = ____

print("Voting Classifier Test Accuracy:", test_acc)
```

MEDIUM

Exercise 2: AdaBoost Iterative Boosting Curve

Train three different `AdaBoostClassifier` models on the training dataset. Set the parameter `n_estimators` to `1`, `10`, and `50` respectively (use `random_state=42`). 1. Fit each model on the training data. 2. Store the trained models in a list called `adaboost_clfs`. 3. Compute the accuracy score of each model on `X_test` and store the scores in a list called `accuracies`.

How to solve: Use the `staged_predict()` method of the trained `AdaBoostClassifier`. Loop through stages to calculate and plot the validation error at each boosting iteration.

TEMPLATE CODE:

```
from sklearn.ensemble import AdaBoostClassifier

# TODO: Fit 3 AdaBoost models for estimators [1, 10, 50] and extract their test set accuracies
adaboost_clfs = []
accuracies = []

print("Accuracies at 1, 10, 50 estimators:", accuracies)
```

HARD

Exercise 3: Two-Stage Stacking Classifier

Implement a basic two-stage stacking pipeline: 1. Fit two base models on `(X_train, y_train)`: - Base Model 1: `KNeighborsClassifier()` - Base Model 2: `SVC(probability=True, random_state=42)` 2. Construct **meta-features** for the test set `X_test` by concatenating the predicted probability of class 1 from each base model. (Hint: extract `predict_proba(X_test)[:, 1]` for both and stack them using `np.column_stack`). Store the resulting $(100, 2)$ matrix in `meta_features`. 3. Fit a meta-classifier `LogisticRegression(random_state=42)` on the training set meta-features (constructed by stacking the predictions of base models on `X_train`). 4. Evaluate the meta-classifier's accuracy on the test meta-features `meta_features` against `y_test`. Store the accuracy score in `test_acc`.

How to solve: Train base classifiers. Use their predicted probabilities on the validation set as features to train a meta-classifier (e.g. Logistic Regression).

TEMPLATE CODE:

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression

# Fit base models
knn = KNeighborsClassifier().fit(X_train, y_train)
svm = SVC(probability=True, random_state=42).fit(X_train, y_train)

# TODO: Build meta_features for test set, train the meta classifier, and
calculate test_acc
meta_features = ____
test_acc = ____

print("Meta-features shape:", meta_features.shape)
print("Stacked model accuracy score:", test_acc)
```

CHAPTER 15

Dimensionality Reduction & Manifold Learning

Theory & Foundations

In machine learning, high-dimensional data (hundreds of features) can cause models to overfit and makes visualization impossible. This is known as the **Curse of Dimensionality**.

1. Linear Reduction Methods

- **PCA (Principal Component Analysis)**: Unsupervised method maximizing data variance.
- **LDA (Linear Discriminant Analysis)**: Supervised method maximizing class separability:

$$S_W = \sum_c S_c \; S_B = \sum_c N_c \, (\mathbf{m}_c - \mathbf{m})(\mathbf{m}_c - \mathbf{m})^T$$

LDA finds a projection that maximizes the ratio of between-class variance to within-class variance (S_B / S_W).

2. Non-linear & Manifold Methods

When data lies on a complex curved manifold, linear projections fail. - **Kernel PCA**: Employs the kernel trick to map data into high-dimensional space where PCA is applied linearly. - **t-SNE (t-Distributed Stochastic Neighbor Embedding)**: Converts Euclidean distances between data points into conditional probabilities representing similarities, preserving local structure for 2D/3D visualizations. - **UMAP (Uniform Manifold Approximation and Projection)**: Similar to t-SNE but faster and preserves more global structure.

Deep Dive: Manifold Mapping and Class Discriminants

High-dimensional datasets suffer from the "curse of dimensionality". Manifold learning finds lower-dimensional projections that preserve the dataset's geometry: - **Linear Discriminant Analysis (LDA)**: A supervised method that projects data onto axes that maximize distance between class means while minimizing variance within classes. - **Kernel PCA**: Uses kernels to project data with non-linear structures. - **t-Distributed Stochastic Neighbor Embedding (t-SNE)**: A non-linear technique that maps high-dimensional datasets to 2D or 3D, keeping similar points close and dissimilar points far apart, ideal for visualizing clusters.

Example Implementation

```
from sklearn.datasets import make_circles
from sklearn.decomposition import KernelPCA
import matplotlib.pyplot as plt

# Generate non-linear concentric circles
X, y = make_circles(n_samples=100, factor=0.5, noise=0.05, random_state=42)

# Fit Kernel PCA
kpca = KernelPCA(n_components=2, kernel='rbf', gamma=15)
X_kpca = kpca.fit_transform(X)

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].scatter(X[:, 0], X[:, 1], c=y, cmap='viridis')
axes[0].set_title("Original Data Space")
axes[1].scatter(X_kpca[:, 0], X_kpca[:, 1], c=y, cmap='viridis')
axes[1].set_title("Kernel PCA Space")
plt.show()
```

Exercises

EASY

Exercise 1: Linear Discriminant Analysis

Perform Linear Discriminant Analysis (`LinearDiscriminantAnalysis`) on the classification dataset `X, y`. 1. Reduce the dataset to **1** dimension (since we have a 2-class classification problem). 2. Store the projected 1D array in `X_lda`.

How to solve: Import `LinearDiscriminantAnalysis`. Train it using `lda.fit(X_train, y_train)`, then project features using `lda.transform(X_train)`.

TEMPLATE CODE:

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=300, n_features=4, n_classes=2, random_s
tate=42)

# TODO: Fit LDA and project X to 1D
X_lda = ____

print("X_lda shape:", X_lda.shape)
```

MEDIUM

Exercise 2: Kernel PCA Projection

The concentric circles dataset is not linearly separable in 2D. Project it using **Kernel PCA** into a new 2D space where the classes are separated. 1. Use `KernelPCA` with **2** components, `'rbf'` kernel, and gamma value **15** (set `random_state=42`). 2. Fit the model and transform `X`. Store the resulting coordinates in `X_kpca`.

How to solve: Use `KernelPCA(n_components=2, kernel='rbf', gamma=15)` to project circular, non-linear data into a linearly separable 2D space.

TEMPLATE CODE:

```
from sklearn.decomposition import KernelPCA
from sklearn.datasets import make_circles

X, y = make_circles(n_samples=300, factor=0.5, noise=0.05, random_state=42)

# TODO: Apply Kernel PCA to project X
X_kpca = ____

print("X_kpca shape:", X_kpca.shape)
```

HARD

Exercise 3: t-SNE vs. PCA Silhouette Score Comparison

Compare t-SNE and PCA projections using the **Silhouette Score** on a cluster dataset. 1. Project the dataset `X` into 2 dimensions using `TSNE` (with `perplexity=30` and `random_state=42`). Compute the silhouette score of the resulting embeddings with respect to the targets `y`. Store the score in `tsne_silhouette`. 2. Project the dataset `X` into 2 dimensions using standard `PCA(n_components=2, random_state=42)`. Compute its silhouette score and store it in `pca_silhouette`. *Hint*: Use `silhouette_score(embeddings, y)` from `sklearn.metrics`.

How to solve: Project data using both PCA and t-SNE. Calculate and compare cluster quality using `silhouette_score(projection, labels)`.

TEMPLATE CODE:

```
from sklearn.manifold import TSNE
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_score
from sklearn.datasets import make_blobs

X, y = make_blobs(n_samples=300, n_features=10, centers=3, cluster_std=3.0, r
andom_state=42)

# TODO: Apply t-SNE and PCA, then calculate their respective silhouette
scores
tsne_silhouette = ____
pca_silhouette = ____

print(f"t-SNE Embeddings Silhouette Score: {tsne_silhouette:.4f}")
print(f"PCA Embeddings Silhouette Score: {pca_silhouette:.4f}")
```

CHAPTER 16

Recommender Systems

Theory & Foundations

Recommender systems aim to predict the rating or preference a user would give to an item, helping users discover new items (books, movies, products).

1. Content-Based Filtering

Recommends items similar to those a user liked in the past. - **Item Profile**: Represents items as vectors of features (e.g. genres, keywords). - **User Profile**: Represents user preferences as a vector in the same feature space. - **Scoring**: Predicted preference is calculated using the dot product or cosine similarity between user and item profiles:

$$\text{Score}(\mathbf{u}, \mathbf{v}) = \mathbf{u} \cdot \mathbf{v}$$

2. Collaborative Filtering (CF)

Recommends items based on the behavior of other similar users. - **User-Based CF**: Find users with similar rating histories. The rating of user u for item i is calculated as a similarity-weighted average of other users' ratings:

$$\hat{r}_{u,\,i} = \frac{\sum_{v \in N} \text{sim}(u,\,v) \cdot r_{v,\,i}}{\sum_{v \in N} |\text{sim}(u,\,v)|}$$

- **Item-Based CF**: Find items similar to item i that user u has previously rated.

3. Latent Factor Models & Matrix Factorization

Decomposes the sparse rating matrix R (dimension $M \times N$) into two lower-rank dense matrices P ($M \times K$, user latent factors) and Q ($N \times K$, item latent factors):

$$R \approx P \cdot Q^{T}$$

We learn P and Q by minimizing the regularized reconstruction error on observed ratings:

$$\min_{P, Q} \sum_{(u, i) \in K} (R_{u, i} - P_u \cdot Q_i^T)^2 + \lambda (\|P_u\|^2 + \|Q_i\|^2)$$

Deep Dive: Latent Factors and Collaborative Filtering

Recommender systems connect users with relevant content: - **Content-Based Filtering**: Recommends items similar to those a user previously interacted with, based on item features. - **Collaborative Filtering**: Recommends items by matching users with similar tastes (User-Based) or identifying items commonly liked together (Item-Based). - **Matrix Factorization**: Decomposes the user-item rating matrix R into user latent factors P and item latent factors Q, such that $R \approx P \cdot Q^T$. These factors are learned iteratively using Stochastic Gradient Descent (SGD) to minimize prediction error.

Example Implementation

```
import numpy as np
import pandas as pd

# Collaborative Filtering ratings matrix example
# Rows = Users, Cols = Items (0 indicates unrated)
ratings = np.array([
    [5, 3, 0, 1],
    [4, 0, 0, 1],
    [1, 1, 0, 5],
    [1, 0, 0, 4],
    [0, 1, 5, 4],
])
print("Ratings Matrix Shape:", ratings.shape)
```

Exercises

EASY

Exercise 1: Content-Based Item Recommendation

Calculate the recommendation scores for a user based on item features. Given item features `item_features` (where columns represent 'Action' and 'Sci-Fi' genre weights) and a user's profile vector `user_profile` (weights of preferences): 1. Compute the dot product between the item features matrix and the user profile vector to calculate scores. 2. Sort the items in descending order of their scores (returning indices of the sorted items). Store the list of indices in `recommendations`.

How to solve: Compute similarity scores: `scores = item_features @ user_profile`. Sort the scores in descending order using `np.argsort(scores)[::-1]` to get recommendations.

TEMPLATE CODE:

```
# Items features: Item 0 (pure action), Item 1 (pure sci-fi), Item 2 (mixed)
item_features = np.array([
    [1.0, 0.0],
    [0.0, 1.0],
    [0.8, 0.2]
])
user_profile = np.array([0.9, 0.1]) # User likes action, dislikes sci-fi

# TODO: Calculate scores and find recommendations index list
recommendations = ____

print("Recommended items indices (highest to lowest score):",
recommendations)
```

MEDIUM

Exercise 2: User-Based Collaborative Filtering Rating Prediction

Calculate the predicted rating of **User 0** for **Item 2** in the ratings matrix below using a similarity-weighted average. 1. The users are: - User 0: `[5.0, 3.0, ?]` (Item 0, Item 1, Item 2) - User 1: `[4.0, ?, 4.0]` - User 2: `[1.0, 1.0, 5.0]` 2. Calculate the cosine similarity between User 0 and the other users based only on their **overlapping rated items** (excluding the target Item 2). - Overlap between User 0 and User 1: Item 0 only. (Cosine Similarity = 1.0) - Overlap between User 0 and User 2: Items 0 and 1. (Compute Cosine Similarity between `[5.0, 3.0]` and `[1.0, 1.0]`). 3. Compute the predicted rating for Item 2 as:

$$\hat{r}_{0,2} = (\text{sim}(0,1) \cdot R(1,2) + \text{sim}(0,2) \cdot R(2,2))/(\text{sim}(0,1) + \text{sim}(0,2))$$

Store your predicted float rating in `predicted_rating`.

How to solve: Calculate cosine similarities between User 0 and other users. Use the similarities as weights to compute the predicted rating for User 0 on Item 2.

TEMPLATE CODE:

```
# Ratings Matrix
ratings_matrix = np.array([
    [5.0, 3.0, 0.0],   # User 0
    [4.0, 0.0, 4.0],   # User 1
    [1.0, 1.0, 5.0]    # User 2
])

# TODO: Compute predicted_rating
predicted_rating = _____

print(f"Predicted rating of User 0 for Item 2: {predicted_rating:.4f}")
```

HARD

Exercise 3: Latent Factor Matrix Factorization from Scratch

Implement the regularized matrix factorization algorithm using Stochastic Gradient Descent (SGD). Write a function `matrix_factorization(R, K=2, steps=5000, alpha=0.002, beta=0.02)` that: 1. Initializes P ($M \times K$) and Q ($N \times K$) randomly with values in $[0, 1]$. Use `np.random.seed(42)`. 2. For each step, loops over all rows u and columns i. If $R[u,i] > 0$ (observed rating): - Computes prediction error:

$$e_{u,i} = R[u, i] - P_u \cdot Q_i^T$$

- Updates latent vectors P_u and Q_i using gradients:

$$P_u \leftarrow P_u + \alpha \cdot (2 \cdot e_{u,i} \cdot Q_i - \beta \cdot P_u)$$

$$Q_i \leftarrow Q_i + \alpha \cdot (2 \cdot e_{u,i} \cdot P_u - \beta \cdot Q_i)$$

3. Returns `(P, Q)` as numpy arrays.

How to solve: Implement the SGD loops: for active ratings, calculate error: `error = R[u,i] - np.dot(P[u], Q[i])`. Update: `P[u] += alpha * (2 * error * Q[i] - beta * P[u])`.

TEMPLATE CODE:

```
def matrix_factorization(R, K=2, steps=5000, alpha=0.002, beta=0.02):
    # TODO: Implement SGD Matrix Factorization
    pass

# Test Matrix Factorization
R = np.array([
    [5, 3, 0, 1],
    [4, 0, 0, 1],
    [1, 1, 0, 5],
    [1, 0, 0, 4],
    [0, 1, 5, 4],
])
P, Q = matrix_factorization(R, K=2, steps=5000)
print("Reconstructed Matrix:
", P @ Q.T)
```

CHAPTER 17

Association Rule Learning

Theory & Foundations

Association rule learning is a rule-based machine learning method for discovering interesting relations (co-occurrences) between variables in large databases. It is widely used in **Market Basket Analysis** to identify products frequently bought together.

1. Metrics for Rule Evaluation

Let A and B be sets of items (itemsets). A rule is written as $A \rightarrow B$ (e.g. $\{milk\} \rightarrow \{bread\}$). - **Support**: Measures how frequently the itemset appears in the database:

$$\text{Support}(A) = (\text{Count}(A))/(\text{Total Transactions } N)$$

- **Confidence**: Measures how often the rule is found to be true. The conditional probability of buying B given A:

$$\text{Confidence}(A \rightarrow B) = (\text{Support}(A \cup B))/(\text{Support}(A))$$

- **Lift**: Measures how much more often A and B occur together than expected if they were statistically independent. A lift > 1 indicates positive correlation:

$$\text{Lift}(A \rightarrow B) = (\text{Confidence}(A \rightarrow B))/(\text{Support}(B))$$

2. Apriori Algorithm

Finding frequent itemsets by checking all combinations is computationally expensive. Apriori uses the **downward closure property**:

> *If an itemset is frequent, all of its subsets must also be frequent.* It generates candidates of size k by joining frequent itemsets of size $k\text{-}1$, pruning candidates containing non-frequent subsets.

Deep Dive: Market Basket Analysis and Association Rules

Association Rule Learning discovers relationships between items in transaction data: - **Support**: Frequency of an itemset:

$$\text{Support}(A \rightarrow B) = (\text{Frequency}(A \cup B))/(\text{Total Transactions})$$

- **Confidence**: Accuracy of the rule:

$$\text{Confidence}(A \rightarrow B) = (\text{Support}(A \cup B))/(\text{Support}(A))$$

- **Lift**: Strength of the rule compared to random chance:

$$\text{Lift}(A \rightarrow B) = (\text{Confidence}(A \rightarrow B))/(\text{Support}(B))$$

A Lift > 1 indicates that items are frequently bought together. - **Apriori Algorithm**: Reduces search space by pruning itemsets whose subsets are infrequent.

Example Implementation

```
# Simple transactions dataset
transactions = [
    ['milk', 'bread'],
    ['milk', 'diaper', 'beer'],
    ['milk', 'bread', 'diaper', 'beer'],
    ['bread', 'diaper', 'beer'],
    ['milk', 'bread', 'diaper']
]
total_transactions = len(transactions)
print("Total Transactions:", total_transactions)
```

Exercises

EASY

Exercise 1: Association Metrics Calculation

Given the database of transactions above, calculate the **Support**, **Confidence**, and **Lift** for the rule:

$$\{milk\} \rightarrow \{bread\}$$

Store your values in variables named `support`, `confidence`, and `lift`.

How to solve: Support is transactions with both items divided by total. Confidence is both items divided by antecedent count. Lift is confidence divided by consequent support.

TEMPLATE CODE:

```
# TODO: Calculate the metrics manually or write code
support = ____
confidence = ____
lift = ____

print(f"Support: {support:.4f}")
print(f"Confidence: {confidence:.4f}")
print(f"Lift: {lift:.4f}")
```

MEDIUM

Exercise 2: Apriori Frequent Itemsets Simulation

Find all frequent itemsets of any size in the dataset `transactions` with a minimum support threshold of `0.4` (i.e. itemsets must appear in at least 2 transactions). Store your frequent itemsets in a list of tuples named `frequent_itemsets` (e.g. `[('milk',), ('bread',), ('milk', 'bread')]`).

How to solve: Join frequent itemsets of size k to generate candidates of size $k+1$. Filter candidates by counting their occurrences in transactions and checking if they meet min support.

TEMPLATE CODE:

```
transactions = [
    ['milk', 'bread'],
    ['milk', 'diaper', 'beer'],
    ['milk', 'bread', 'diaper', 'beer'],
    ['bread', 'diaper', 'beer'],
    ['milk', 'bread', 'diaper']
]

# TODO: Identify all itemsets with support >= 0.4.
# Store frequent itemsets as a list of tuples.
frequent_itemsets = _____

print("Frequent Itemsets count:", len(frequent_itemsets))
print("Itemsets list:", frequent_itemsets)
```

HARD

Exercise 3: Association Rule Generator

Write a function `generate_rules(frequent_itemsets_with_support, min_confidence=0.7, min_lift=1.1)` that extracts association rules. - Input `frequent_itemsets_with_support` is a dictionary where keys are itemset tuples and values are their support floats. - The function must look at every frequent itemset of size ≥ 2. - For each itemset, consider all possible non-empty proper subsets A. The remaining items form consequent B. - Calculate:

$$\text{Confidence}(A \to B) = (\text{Support}(A \cup B))/(\text{Support}(A))$$

$$\text{Lift}(A \to B) = (\text{Confidence}(A \to B))/(\text{Support}(B))$$

- Filter and return rules where confidence $\geq \min\_confidence$ and lift $\geq \min\_lift$ as a list of tuples: `(antecedent_tuple, consequent_tuple, confidence, lift)`.

How to solve: For each frequent itemset, generate possible rules, calculate confidence and lift, and filter by thresholds.

TEMPLATE CODE:

```
def generate_rules(frequent_itemsets_with_support, min_confidence=0.7, min_li
ft=1.1):
    # TODO: Generate rules matching thresholds
    rules = []
    return rules

# Sample frequent itemsets dictionary with supports
itemsets_with_support = {
    ('milk',): 0.8,
    ('bread',): 0.8,
    ('diaper',): 0.8,
    ('beer',): 0.6,
    ('milk', 'bread'): 0.6,
    ('milk', 'diaper'): 0.6,
    ('milk', 'beer'): 0.4,
    ('bread', 'diaper'): 0.6,
    ('bread', 'beer'): 0.4,
    ('diaper', 'beer'): 0.6,
    ('milk', 'diaper', 'beer'): 0.4,
    ('bread', 'diaper', 'beer'): 0.4
}
rules_generated = generate_rules(itemsets_with_support)
print("Generated rules count:", len(rules_generated))
```

CHAPTER 18

Semi-Supervised Learning

Theory & Foundations

Semi-supervised learning is a branch of machine learning that combines a small amount of labeled data with a large amount of unlabeled data during training. This is extremely useful because labeling data can be expensive and time-consuming, while unlabeled data is often cheap and abundant.

1. Label Propagation and Label Spreading

Graph-based algorithms that model the data points as nodes in a connected graph. - **Edges**: Weighted by similarity between points. - **Propagation**: Labels propagate from labeled nodes along the edges to unlabeled nodes. - **Label Spreading**: A variant that uses a normalized Laplacian matrix and adds a regularization term to make it robust to noise.

2. Pseudo-Labeling (Self-Training)

An iterative wrapper-method: 1. Train a classifier on the small set of labeled data. 2. Predict class probabilities for the unlabeled data. 3. Select unlabeled samples whose predicted probability exceeds a high threshold (e.g. ≥ 0.9). 4. Add these samples to the training set as labeled, using their predicted label ("pseudo-label"). 5. Retrain the classifier on the expanded training set and repeat.

Deep Dive: Graph Label Propagation and Self-Training

Semi-Supervised learning utilizes a small labeled dataset and a large unlabeled dataset: - **Label Propagation**: Builds a graph where data points are nodes and edges represent similarity. Labels are propagated from labeled nodes to unlabeled neighbors based on edge weights. - **Self-Training (Pseudo-Labeling)**: Trains a model on labeled data, predicts probabilities for unlabeled data, adds predictions that exceed a confidence threshold to the training set, and retrains the model. - **Label Spreading**: A variant of Label Propagation that uses a normalized Laplacian matrix and adds regularization to prevent noise propagation.

Example Implementation

```
from sklearn.semi_supervised import LabelPropagation
from sklearn.datasets import make_classification
import numpy as np

# Make data where -1 denotes unlabeled points
X, y = make_classification(n_samples=100, n_features=4, n_classes=2,
random_state=42)
y_masked = y.copy()
y_masked[80:] = -1 # Mask 20% labels as unlabeled

lp = LabelPropagation().fit(X, y_masked)
print("Transduced Labels for unlabeled points:", lp.transduction_[80:])
```

Exercises

EASY

Exercise 1: Graph-based Label Propagation

Train a `LabelPropagation` model on the dataset where **90%** of labels are masked as unlabeled (`-1`). 1. Use `kernel='knn'` and `n_neighbors=5`. 2. Fit the model on training features `X_train` and labels `y_train` (containing `-1` masks). 3. Compute the accuracy score of the propagated labels on the test mask items. Store your fitted model object in `lp_model` and the accuracy score in `acc`.

How to solve: Train `LabelPropagation(kernel='knn', n_neighbors=5)`. Evaluate accuracy on the unlabeled indices by comparing model predictions with true labels.

TEMPLATE CODE:

```
from sklearn.semi_supervised import LabelPropagation
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

X, y = make_classification(n_samples=500, n_features=6, n_classes=2, random_s
tate=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
dom_state=42)

# Mask 90% of training labels
np.random.seed(42)
mask = np.random.rand(len(y_train)) < 0.9
y_train_masked = y_train.copy()
y_train_masked[mask] = -1

# TODO: Train LabelPropagation model and evaluate accuracy on test set
lp_model = ____
acc = ____

print(f"Transduction Accuracy: {acc:.4f}")
```

MEDIUM

Exercise 2: Self-Training Pseudo-Labeling Loop

Implement a single iteration of Pseudo-Labeling. Write a function `pseudo_labeling(X, y_masked, threshold=0.9)` that: 1. Splits the training set into labeled indices (where `y_masked != -1`) and unlabeled indices (where `y_masked == -1`). 2. Trains a `LogisticRegression` classifier on the labeled data. 3. Predicts probabilities for the unlabeled data. 4. Identifies samples where the predicted probability of the predicted class is strictly greater than or equal to `threshold`. 5. Adds these highly confident samples and their predicted labels to the labeled dataset. 6. Retrains a final `LogisticRegression` classifier on the combined dataset and returns this fitted classifier object.

How to solve: Fit classifier on labeled data. Predict probabilities on unlabeled data. Find samples exceeding the confidence threshold, add them to the training set, and repeat.

TEMPLATE CODE:

```
from sklearn.linear_model import LogisticRegression

def pseudo_labeling(X, y_masked, threshold=0.9):
    # TODO: Implement pseudo-labeling iteration
    pass

# Test on a small set
X_mock = np.random.randn(10, 2)
y_mock = np.array([0, 1, 0, 1, 0, -1, -1, -1, -1, -1])
clf = pseudo_labeling(X_mock, y_mock)
print("Classifier trained successfully:", clf)
```

HARD

Exercise 3: Label Spreading Kernel Hyperparameter Tuning

Evaluate the impact of the Radial Basis Function (RBF) kernel `gamma` parameter on the `LabelSpreading` model. The concentric moons dataset is non-linear. Evaluate `gamma` values `[0.1, 1.0, 10.0, 100.0]`. 1. For each gamma, fit `LabelSpreading(kernel='rbf', gamma=gamma)` on the training set `(X_train, y_train_masked)`. 2. Compute the accuracy of the propagated labels against the actual labels of the test mask indices (where `y_train_masked == -1`). 3. Store the best gamma found in `best_gamma` and its corresponding accuracy in `best_acc`.

How to solve: Perform a grid search over gamma values. Train `LabelSpreading(kernel='rbf', gamma=g)` for each value and select the gamma that yields the highest test accuracy.

TEMPLATE CODE:

```
from sklearn.semi_supervised import LabelSpreading
from sklearn.datasets import make_moons

X, y = make_moons(n_samples=500, noise=0.15, random_state=42)
y_masked = y.copy()
np.random.seed(42)
mask = np.random.rand(len(y)) < 0.8
y_masked[mask] = -1 # Mask 80% labels

# TODO: Loop over gammas, train LabelSpreading, find best_gamma and best_acc
best_gamma = 0.0
best_acc = 0.0

print(f"Best Gamma: {best_gamma} with Test Mask Accuracy: {best_acc:.4f}")
```

CHAPTER 19

Classification on Imbalanced Data

Theory & Foundations

Class imbalance occurs when the classes in a classification dataset are not represented equally. For example, in fraud detection, only 0.1% of transactions may be fraudulent.

1. The Accuracy Metric Trap

When classes are highly imbalanced, **accuracy** is a misleading metric. If 99% of samples are of class 0, a naive model predicting 0 for all samples achieves 99% accuracy but fails completely. We must evaluate models using: - **Precision**: $(\mathrm{TP})/(\mathrm{TP} + \mathrm{FP})$ (Fraction of predicted positives that are correct). - **Recall**: $(\mathrm{TP})/(\mathrm{TP} + \mathrm{FN})$ (Fraction of actual positives found). - **F1 Score**: $2 \cdot (\mathrm{Precision} \cdot \mathrm{Recall})/(\mathrm{Precision} + \mathrm{Recall})$ (Harmonic mean). - **Macro F1 Score**: Averaging the F1 scores calculated separately for each class.

2. Resampling Techniques

- **Oversampling**: Duplicating minority class samples or synthesizing new ones (**SMOTE**).
- **Undersampling**: Removing majority class samples to balance frequencies.

3. Cost-Sensitive Learning

Instead of balancing the datasets, we modify the loss function during training, assigning a higher weight (penalty) to misclassifications of the minority class. In scikit-learn, this is set using the parameter `class_weight='balanced'`.

Deep Dive: Imbalanced Distribution Remediation

Standard machine learning models assume balanced class distributions. With highly imbalanced data (e.g., fraud detection), a model can achieve 99% accuracy by predicting the majority class. Remedies include: - **Evaluation Metrics**: Evaluate macro F1-score, precision, recall, or Area Under ROC curve instead of accuracy. - **Oversampling**: Replicates minority class instances (e.g., Random Oversampling, SMOTE) to balance the distribution. - **Cost-Sensitive Learning**: Penalizes misclassifying the minority class more heavily by adjusting loss function weights (e.g., setting `class_weight='balanced'`).

Example Implementation

```
from sklearn.datasets import make_classification
from sklearn.metrics import classification_report
from sklearn.svm import SVC
import numpy as np

# Highly imbalanced dataset
X, y = make_classification(n_samples=1000, weights=[0.95, 0.05], random_state=42)
print("Class distribution:", np.bincount(y))

# Train classifier
clf = SVC(kernel='linear', random_state=42).fit(X, y)
print("Classification Report:")
print(classification_report(y, clf.predict(X)))
```

Exercises

EASY

Exercise 1: Accuracy vs. Macro F1 Metric Trap

Calculate the overall **accuracy** and the **macro F1-score** of a baseline classifier that predicts class 0 for all samples. The dataset has **100** samples: - True labels: **95** are class `0`, and **5** are class `1`. - Model predictions: all **100** predictions are class `0`.

Store your answers in `accuracy` and `f1`.

How to solve: Calculate the classification accuracy and the macro-averaged F1-score: `f1_score(y_true, y_pred, average='macro')`. Note how accuracy remains high while F1 drops.

TEMPLATE CODE:

```
# TODO: Calculate accuracy and macro F1 score
accuracy = _____
f1 = _____

print(f"Accuracy: {accuracy:.4f}")
print(f"Macro F1-Score: {f1:.4f}")
```

MEDIUM

Exercise 2: Random Oversampling from Scratch

Implement the Random Oversampling algorithm. Write a function `random_oversample(X, y)` that: 1. Identifies the size of the majority class (N_{maj}) and minority class (N_{min}). 2. Randomly selects and duplicates samples from the minority class (with replacement) so that its count matches N_{maj}. 3. Combines the original majority class samples with the expanded minority class samples. 4. Returns the balanced features and labels arrays `(X_resampled, y_resampled)`.

How to solve: Separate majority and minority indices. Sample minority indices with replacement using `np.random.choice()` to match the majority size. Concatenate indices.

TEMPLATE CODE:

```
def random_oversample(X, y):
    # TODO: Implement random oversampling
    pass

# Test oversampler
X_test = np.array([[1, 2], [3, 4], [5, 6]])
y_test = np.array([0, 0, 1])
X_res, y_res = random_oversample(X_test, y_test)
print("Resampled labels:", y_res)
```

HARD

Exercise 3: Cost-Sensitive SVM Classification

Apply cost-sensitive learning to train a Support Vector Classifier (`SVC`) on an imbalanced set. 1. Train a linear Support Vector Classifier `SVC(kernel='linear', random_state=42)` on `X_train` and `y_train` using **cost-sensitive weights** (set the parameter `class_weight='balanced'`). 2. Evaluate the classifier's performance by computing its **macro F1-score** on the test set `X_test`. Store the score in `cost_sensitive_score`.

How to solve: Train an SVM classifier, setting the hyperparameter `class_weight='balanced'` to penalize minority misclassifications more heavily.

TEMPLATE CODE:

```
from sklearn.svm import SVC
from sklearn.metrics import f1_score
from sklearn.model_selection import train_test_split

X, y = make_classification(n_samples=1000, n_features=6, weights=[0.92,
0.08], random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, ran
dom_state=42)

# TODO: Fit cost-sensitive SVC and calculate macro F1 score on test set
cost_sensitive_score = ____

print("Cost-Sensitive Classifier Macro F1 Score:", cost_sensitive_score)
```

CHAPTER 20

Hyperparameter Optimization with Optuna

Theory & Foundations

Tuning hyperparameters (learning rate, tree depth, regularization strength) is critical to squeeze out model performance.

1. Optimization Strategies

- **Grid Search**: Evaluates all combinations in a user-defined grid. Slow and suffers from the curse of dimensionality.
- **Random Search**: Samples combinations randomly. Faster and often finds better configurations than Grid Search in fewer iterations.
- **Bayesian Optimization**: Fits a probabilistic surrogate model (e.g. Gaussian Process) to the history of evaluations, using an **Acquisition Function** to balance exploration and exploitation to select the next parameter to evaluate.

2. Optuna Framework

Optuna is a state-of-the-art framework for hyperparameter optimization: - **Study**: Optimization session. Created using `optuna.create_study()`. - **Objective Function**: Takes a `trial` object, suggests parameters using methods like `trial.suggest_float()` or `trial.suggest_int()`, evaluates the model, and returns the validation metric. - **Trial**: Single evaluation of the objective function.

Deep Dive: Probabilistic Hyperparameter Tuning

Hyperparameters control the model learning process. Traditional tuning methods are uninformed: Grid Search is slow, and Random Search is random. **Bayesian Optimization** treats hyperparameter tuning as a black-box optimization problem. It builds a probabilistic surrogate model (usually a Gaussian Process) to model hyperparameter performance. It uses an **Acquisition Function** (like Upper Confidence Bound, UCB) to decide which hyperparameters to evaluate next, balancing exploration (sampling uncertain regions) and exploitation (sampling promising regions). **Optuna** is a framework that automates this process using tree-structured Parzen estimators.

Example Implementation

```
import optuna
import logging

# Disable optuna logs to avoid clutter
optuna.logging.set_verbosity(optuna.logging.WARNING)

# Simple optimization demo: find x to minimize (x - 4)^2
def objective(trial):
    x = trial.suggest_float('x', -10, 10)
    return (x - 4) ** 2

study = optuna.create_study(direction='minimize')
study.optimize(objective, n_trials=50)
print("Best Parameter:", study.best_params)
print("Best Value:", study.best_value)
```

Exercises

EASY

Exercise 1: Optuna 2D Optimization

Write an Optuna study to find the minimum of the mathematical function:

$$f(x, y) = (x - 2)^2 + (y - 3)^2$$

1. Suggest `x` as a float parameter in [-10, 10].
2. Suggest `y` as a float parameter in [-10, 10].
3. Optimize the function for **30 trials**. Store your study object in `study`.

How to solve: In the objective function, suggest values for x and y. Return the quadratic formula. Create a study with `optuna.create_study()` and optimize it.

TEMPLATE CODE:

```
import optuna

# TODO: Create study and optimize objective function
study = ____

print("Best Parameters:", study.best_params)
```

MEDIUM

Exercise 2: Tuning Gradient Boosting Classifiers with Optuna

Optimize three hyperparameters of a `GradientBoostingClassifier` on the classification dataset: 1. `'max_depth'`: suggest integers between `2` and `6` (inclusive). 2. `'n_estimators'`: suggest integers between `10` and `100` (inclusive). 3. `'learning_rate'`: suggest floats between `0.01` and `0.2` (inclusive). 4. Run your optimization study for **20 trials** using `random_state=42` for the classifier. Store the best parameters dictionary in `best_params` and the best accuracy value in `best_value`.

How to solve: In the objective, suggest hyperparameters for a Gradient Boosting classifier. Fit the model, calculate accuracy on test data, and return this value. Optimize study.

TEMPLATE CODE:

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=500, n_features=8, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, ran
dom_state=42)

# TODO: Write objective function and run Optuna study
best_params = ____
best_value = ____

print("Best Parameters found:", best_params)
print(f"Best Test Accuracy: {best_value:.4f}")
```

HARD

Exercise 3: Bayesian Optimization from Scratch

Implement a basic Bayesian Optimization loop using a **Gaussian Process surrogate model** to find the maximum of a 1D function $f(x)$ within bounds. Write a function `bayesian_optimization(objective, bounds, n_iters=10)` that: 1. Randomly samples 3 initial points from the search space `bounds` (1D bounds array of shape `(1, 2)`). Use `np.random.seed(42)` and evaluate the `objective` at these points. 2. For `n_iters` iterations: - Fits a `GaussianProcessRegressor(kernel=Matern(nu=2.5), random_state=42)` on the evaluated points `X` and targets `y`. - Computes prediction mean μ and standard deviation σ over a dense grid of 100 points in `bounds` (constructed using `np.linspace`). - Calculates the acquisition function values:

$$\text{Acquisition}(x) = \mu(x) + 1.96 \cdot \sigma(x)$$

- Finds the next point x_{next} that maximizes this acquisition value. - Evaluates the `objective` at x_{next} and appends `(x_next, y_next)` to the dataset. 3. Returns the overall best coordinate `(best_x, best_y)` corresponding to the maximum objective value found.

How to solve: Fit a Gaussian Process on visited points. Calculate Upper Confidence Bound (mean + 1.96 * std) over a dense grid. Query the point that maximizes UCB, evaluate objective, and repeat.

TEMPLATE CODE:

```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import Matern

def bayesian_optimization(objective, bounds, n_iters=10):
    # TODO: Implement Gaussian Process Bayesian Optimization loop
    pass

# Test 1D Optimization (finding maximum)
def f(x):
    # Maximum is at x = 1.5
    return -(x[0] - 1.5)**2 + 4.0

bounds_1d = np.array([[0.0, 3.0]])
best_coord, max_val = bayesian_optimization(f, bounds_1d, n_iters=10)
print(f"Optimal coordinate: {best_coord[0]:.4f} with Value: {max_val:.4f}")
```

APPENDIX

Exercise Solutions

This appendix contains hints and reference code implementations for solving every exercise in this curriculum.

Chapter 1: Introduction & Data Manipulation (Pandas & NumPy) Solutions

Exercise 1: Filter and Select (Easy)

Hint: Use df[df['Age'] > 30][['Name', 'Score']]

Reference Implementation:

```
result_df = df[df['Age'] > 30][['Name', 'Score']]
```

Exercise 2: Groupby and Aggregations (Medium)

Hint: Use sales_df.groupby('Category').agg(Avg_Revenue=('Revenue', 'mean'), Total_Quantity=('Quantity', 'sum'))

Reference Implementation:

```
agg_df = sales_df.groupby('Category').agg(
    Avg_Revenue=('Revenue', 'mean'),
    Total_Quantity=('Quantity', 'sum')
)
```

Exercise 3: Matrix Row Normalization and Multiplication (Hard)

Hint: Use np.sum(A, axis=1, keepdims=True) to compute row sums. Divide A by these sums to normalize. Then use np.dot or @ for multiplication.

Reference Implementation:

```
def normalize_and_multiply(A, B):
    row_sums = np.sum(A, axis=1, keepdims=True)
    A_norm = A / row_sums
    return A_norm @ B
```

Chapter 2: Data Cleaning & Preprocessing Solutions

Exercise 1: Missing Value Imputation (Easy)

Hint: Use s.fillna(s.mean()) to replace NaNs with the average of the series.

Reference Implementation:

```
s_imputed = s.fillna(s.mean())
```

Exercise 2: Ordinal & One-Hot Encoding (Medium)

Hint: Map 'Size' with a dictionary, and use pd.get_dummies(df, columns=['Color']) for one-hot encoding.

Reference Implementation:

```
df['Size_Encoded'] = df['Size'].map({'Small': 0, 'Medium': 1, 'Large': 2})
encoded_df = pd.get_dummies(df, columns=['Color'])
```

Exercise 3: Custom Scaling with Percentile Clipping (Hard)

Hint: Calculate percentiles using np.percentile(arr, 10) and np.percentile(arr, 90). Clip the array using np.clip. Then do min-max scaling.

Reference Implementation:

```
def custom_scale(arr, min_val, max_val):
    p10 = np.percentile(arr, 10)
    p90 = np.percentile(arr, 90)
    clipped = np.clip(arr, p10, p90)
    scaled = (clipped - clipped.min()) / (clipped.max() - clipped.min()) *
(max_val - min_val) + min_val
    return scaled
```

Chapter 3: Data Visualization & Plotting Solutions

Exercise 1: Plotting a Distribution Histogram (Easy)

Hint: Use plt.hist(data) or sns.histplot(data). Remember to add plt.title(), plt.xlabel(), plt.ylabel().

Reference Implementation:

```
plt.figure(figsize=(8, 5))
plt.hist(data, bins=20, color='skyblue', edgecolor='black')
plt.title('Feature Distribution')
plt.xlabel('Value')
plt.ylabel('Frequency')
plt.grid(True)
plt.show()
```

Exercise 2: Scatter Plot with Regression Line and Hue (Medium)

Hint: Use sns.scatterplot or sns.regplot. For a regression line with hue, sns.lmplot(x='x', y='y', hue='group', data=df) is ideal.

Reference Implementation:

```
sns.lmplot(x='x', y='y', hue='group', data=df, height=6, aspect=1.2)
plt.title('Relationship between X and Y by Group')
plt.show()
```

Exercise 3: Filtered Correlation Heatmap (Hard)

Hint: Compute the correlation matrix using df.corr(). Construct a mask for weak correlations or simply mask them out. Pass it to sns.heatmap.

Reference Implementation:

```
corr = df.corr()
# Mask out weak correlations
corr_filtered = corr.copy()
corr_filtered[corr_filtered.abs() < 0.3] = np.nan
sns.heatmap(corr_filtered, annot=True, cmap='coolwarm', fmt='.2f', vmin=-1, vmax=1)
plt.title('Correlation Heatmap (|r| >= 0.3)')
plt.show()
```

Chapter 4: Regression (Linear & Logistic) Solutions

Exercise 1: Fit a Linear Regression Model (Easy)

Hint: Instantiate with model = LinearRegression(), train with model.fit(X_train, y_train), then make predictions with model.predict(X_test).

Reference Implementation:

```
model = LinearRegression()
model.fit(X_train, y_train)
preds = model.predict(X_test)
```

Exercise 2: Gradient Descent from Scratch (Medium)

Hint: Initialize w and b to 0. Loop over epochs. Calculate predictions: pred = w*X + b. Calculate gradients: dw = -2/N * sum(X * (y - pred)), db = -2/N * sum(y - pred). Update w = w - lr*dw, b = b - lr*db.

Reference Implementation:

```
def gradient_descent(X, y, lr, epochs):
    w, b = 0.0, 0.0
    N = len(X)
    for _ in range(epochs):
        pred = w * X + b
        dw = -2/N * np.sum(X * (y - pred))
        db = -2/N * np.sum(y - pred)
        w -= lr * dw
        b -= lr * db
    return w, b
```

Exercise 3: Regularization Comparison (Hard)

Hint: Train LogisticRegression(penalty='l1', solver='liblinear', C=0.1) for L1 and LogisticRegression(penalty='l2', C=0.1) for L2. Extract coef_[0] for each.

Reference Implementation:

```
model_l1 = LogisticRegression(penalty='l1', solver='liblinear', C=0.1,
random_state=42).fit(X_train, y_train)
model_l2 = LogisticRegression(penalty='l2', C=0.1, random_state=42).fit(X_train, y_
train)
coef_l1 = model_l1.coef_[0]
coef_l2 = model_l2.coef_[0]
```

Chapter 5: Decision Trees & Random Forests Solutions

Exercise 1: Train a Constrained Decision Tree (Easy)

Hint: Instantiate DecisionTreeClassifier(max_depth=3, random_state=42) and fit on X_train, y_train.

Reference Implementation:

```
dt_model = DecisionTreeClassifier(max_depth=3, random_state=42)
dt_model.fit(X_train, y_train)
```

Exercise 2: Random Forest Ensemble Tuning (Medium)

Hint: Instantiate RandomForestClassifier(n_estimators=100, max_depth=5, min_samples_split=4, random_state=42), fit it, predict on X_test, and compute accuracy_score(y_test, preds).

Reference Implementation:

```
rf_model = RandomForestClassifier(n_estimators=100, max_depth=5,
min_samples_split=4, random_state=42)
rf_model.fit(X_train, y_train)
preds = rf_model.predict(X_test)
rf_acc = accuracy_score(y_test, preds)
```

Exercise 3: Feature Importances & Selection (Hard)

Hint: Get importances from rf_model.feature_importances_. Use np.where(importances > 0.1)[0] for indices. Then index X_train[:, selected_indices].

Reference Implementation:

```
importances = rf_model.feature_importances_
selected_indices = np.where(importances > 0.1)[0]
X_train_filtered = X_train[:, selected_indices]
```

Chapter 6: Support Vector Machines & Naive Bayes Solutions

Exercise 1: Naive Bayes Text Classification (Easy)

Hint: Instantiate MultinomialNB(alpha=1.0) and fit on X_train, y_train.

Reference Implementation:

```
nb_model = MultinomialNB(alpha=1.0)
nb_model.fit(X_train, y_train)
```

Exercise 2: SVM Support Vectors (Medium)

Hint: Instantiate SVC(kernel='linear', C=1.0, random_state=42), fit it, and assign support_indices = svm_model.support_.

Reference Implementation:

```
svm_model = SVC(kernel='linear', C=1.0, random_state=42)
svm_model.fit(X_train, y_train)
support_indices = svm_model.support_
```

Exercise 3: Linear vs. RBF Kernels on Non-linear Data (Hard)

Hint: Fit SVC(kernel='linear', random_state=42) and SVC(kernel='rbf', random_state=42) on X_train, y_train. Compute and return their accuracies on X_test.

Reference Implementation:

```
svm_lin = SVC(kernel='linear', random_state=42).fit(X_train, y_train)
svm_rbf = SVC(kernel='rbf', random_state=42).fit(X_train, y_train)
acc_linear = accuracy_score(y_test, svm_lin.predict(X_test))
acc_rbf = accuracy_score(y_test, svm_rbf.predict(X_test))
```

Chapter 7: Unsupervised Learning & Clustering Solutions

Exercise 1: Dimension Reduction with PCA (Easy)

Hint: Use PCA(n_components=2, random_state=42).fit_transform(X).

Reference Implementation:

```
pca = PCA(n_components=2, random_state=42)
X_pca = pca.fit_transform(X)
```

Exercise 2: K-Means & The Elbow Curve (Medium)

Hint: Loop from k=1 to 5. Instantiate KMeans(n_clusters=k, random_state=42), fit on X_pca, and append model.inertia_ to your list.

Reference Implementation:

```
inertias = []
for k in range(1, 6):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X_pca)
    inertias.append(kmeans.inertia_)
```

Exercise 3: K-Means Clustering from Scratch (Hard)

Hint: Initialize centers by randomly picking k samples from X. In the loop, assign labels by calculating distance to all centers (e.g. np.linalg.norm(X[:, np.newaxis] - centers, axis=2).argmin(axis=1)). Then recalculate centers as mean of each group.

Reference Implementation:

```
def kmeans_scratch(X, k, max_iters=100):
    np.random.seed(42)
    idx = np.random.choice(len(X), k, replace=False)
    centers = X[idx]
    for _ in range(max_iters):
        # Distances to all centers
        dists = np.linalg.norm(X[:, np.newaxis] - centers, axis=2)
        labels = np.argmin(dists, axis=1)
        # Update centers
        new_centers = np.array([X[labels == i].mean(axis=0) if len(X[labels == i])
> 0 else centers[i] for i in range(k)])
        if np.allclose(centers, new_centers):
            break
        centers = new_centers
    return centers, labels
```

Chapter 8: Feature Engineering & Selection Solutions

Exercise 1: Polynomial and Interaction Features (Easy)

Hint: Use PolynomialFeatures(degree=2, include_bias=False).fit_transform(X).

Reference Implementation:

```
poly = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly.fit_transform(X)
```

Exercise 2: Cyclical Time Feature Extraction (Medium)

Hint: Get hours using df['timestamp'].dt.hour. Calculate sine and cosine using 2 * np.pi * hours / 24.

Reference Implementation:

```
hours = df['timestamp'].dt.hour
df['hour_sin'] = np.sin(2 * np.pi * hours / 24.0)
df['hour_cos'] = np.cos(2 * np.pi * hours / 24.0)
```

Exercise 3: Recursive Feature Elimination (Hard)

Hint: Instantiate RFE(estimator=RandomForestClassifier(random_state=42), n_features_to_select=3). Fit it on X, y. Get X_selected using rfe_model.transform(X), and ranking from rfe_model.ranking_.

Reference Implementation:

```
rfe_model = RFE(estimator=RandomForestClassifier(random_state=42), n_features_to_se
lect=3)
rfe_model.fit(X, y)
X_selected = rfe_model.transform(X)
ranking = rfe_model.ranking_
```

Chapter 9: Model Evaluation & Hyperparameter Tuning Solutions

Exercise 1: K-Fold Cross-Validation (Easy)

Hint: Instantiate KFold(5, shuffle=True, random_state=42), run cross_val_score(rf, X, y, cv=kf), and call .mean() on the output.

Reference Implementation:

```
kf = KFold(5, shuffle=True, random_state=42)
mean_cv_score = cross_val_score(rf, X, y, cv=kf).mean()
```

Exercise 2: Grid Search vs. Randomized Search Tuning (Medium)

Hint: Define param_grid = {'max_depth': [3, 5, 7], 'min_samples_split': [2, 5, 10]}. Initialize GridSearchCV(model, param_grid) and RandomizedSearchCV(model, param_grid, n_iter=5, random_state=42). Fit both, and extract .best_params_.

Reference Implementation:

```
param_grid = {'max_depth': [3, 5, 7], 'min_samples_split': [2, 5, 10]}
grid_search = GridSearchCV(DecisionTreeRegressor(random_state=42), param_grid,
cv=3).fit(X, y)
random_search = RandomizedSearchCV(DecisionTreeRegressor(random_state=42), param_g
rid, n_iter=5, cv=3, random_state=42).fit(X, y)
best_params_grid = grid_search.best_params_
best_params_random = random_search.best_params_
```

Exercise 3: F1-Score Classification Threshold Tuning (Hard)

Hint: Loop threshold values in np.arange(0.1, 0.91, 0.01). Calculate predictions: (y_probs >= t).astype(int). Calculate f1_score(y_test, predictions). Track and update the maximum F1 and its corresponding threshold.

Reference Implementation:

```
best_threshold = 0.0
max_f1 = 0.0
for t in np.arange(0.1, 0.91, 0.01):
    preds = (y_probs >= t).astype(int)
    score = f1_score(y_test, preds)
    if score > max_f1:
        max_f1 = score
        best_threshold = t
```

Chapter 10: Introduction to Deep Learning Solutions

Exercise 1: Multi-Layer Perceptron Classifier (Easy)

Hint: Instantiate MLPClassifier(hidden_layer_sizes=(64, 32), activation='relu', random_state=42) and fit on X_train, y_train.

Reference Implementation:

```
mlp_model = MLPClassifier(hidden_layer_sizes=(64, 32), activation='relu', random_st
ate=42)
mlp_model.fit(X_train, y_train)
```

Exercise 2: Perceptron from Scratch (Medium)

Hint: predict_fn: 1 if np.dot(x, weights) + bias >= 0 else 0. train_fn: Loop epochs and samples. If prediction != target, update w += lr * (target - pred) * x, and b += lr * (target - pred).

Reference Implementation:

```
def perceptron_predict(x, weights, bias):
    return 1 if np.dot(x, weights) + bias >= 0 else 0

def perceptron_train(X, y, lr=0.1, epochs=100):
    weights = np.zeros(X.shape[1])
    bias = 0.0
    for _ in range(epochs):
        for xi, yi in zip(X, y):
            pred = perceptron_predict(xi, weights, bias)
            error = yi - pred
            weights += lr * error * xi
            bias += lr * error
    return weights, bias
```

Exercise 3: Two-Layer Neural Network from Scratch (Hard)

Hint: Implement sigmoid and its derivative. Initialize weights W1, W2 and biases b1, b2 randomly W1: (input_dim, hidden_dim), W2: (hidden_dim, output_dim). Forward: h = sigmoid(X @ W1 + b1), out = sigmoid(h @ W2 + b2). Backward: delta2 = (out - y) * out * (1 - out), delta1 = (delta2 @ W2.T) * h * (1 - h). Gradients: dW2 = h.T @ delta2, db2 = sum(delta2), dW1 = X.T @ delta1, db1 = sum(delta1). Update parameters.

Reference Implementation:

```
def sigmoid(x):
    return 1 / (1 + np.exp(-x))

def sigmoid_derivative(x):
    return x * (1 - x)

class SimpleNN:
    def __init__(self, input_dim, hidden_dim, output_dim):
        np.random.seed(42)
        self.W1 = np.random.randn(input_dim, hidden_dim) * 0.1
        self.b1 = np.zeros((1, hidden_dim))
        self.W2 = np.random.randn(hidden_dim, output_dim) * 0.1
        self.b2 = np.zeros((1, output_dim))

    def forward(self, X):
        self.z1 = np.dot(X, self.W1) + self.b1
        self.a1 = sigmoid(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = sigmoid(self.z2)
        return self.a2

    def backward(self, X, y, lr):
        m = X.shape[0]
        da2 = self.a2 - y
        dz2 = da2 * sigmoid_derivative(self.a2)
        dW2 = np.dot(self.a1.T, dz2) / m
        db2 = np.sum(dz2, axis=0, keepdims=True) / m

        da1 = np.dot(dz2, self.W2.T)
        dz1 = da1 * sigmoid_derivative(self.a1)
        dW1 = np.dot(X.T, dz1) / m
        db1 = np.sum(dz1, axis=0, keepdims=True) / m

        self.W2 -= lr * dW2
        self.b2 -= lr * db2
        self.W1 -= lr * dW1
        self.b1 -= lr * db1

    def train(self, X, y, lr=0.1, epochs=10000):
        for _ in range(epochs):
            self.forward(X)
            self.backward(X, y, lr)
```

Chapter 11: Time Series Analysis & Forecasting Solutions

Exercise 1: Rolling Statistics (Easy)

Hint: Calculate rolling statistics using df['value'].rolling(window=7).mean() and df['value'].rolling(window=7).std().

Reference Implementation:

```
rolling_mean = df['value'].rolling(window=7).mean()
rolling_std = df['value'].rolling(window=7).std()
```

Exercise 2: Time Series Differencing & Stationarity Test (Medium)

Hint: Use df['value'].diff().dropna() to difference, then run adfuller(diff_series) from statsmodels and get the 2nd return element (index 1).

Reference Implementation:

```
from statsmodels.tsa.stattools import adfuller
diff_series = df['value'].diff().dropna()
adf_pvalue = adfuller(diff_series)[1]
```

Exercise 3: Time Series Forecasting and MAPE (Hard)

Hint: Fit ARIMA(train, order=(1,1,0)). Forecast steps=20. Calculate mean(abs(test - forecast)/test) * 100.

Reference Implementation:

```
from statsmodels.tsa.arima.model import ARIMA
model = ARIMA(train, order=(1,1,0)).fit()
fc = model.forecast(steps=20)
mape = np.mean(np.abs((test['value'] - fc) / test['value'])) * 100
```

Chapter 12: Natural Language Processing (NLP) Solutions

Exercise 1: Basic Text Preprocessing & Token Count (Easy)

Hint: Lower the text, remove punctuation, split, filter out stopwords, and count.

Reference Implementation:

```
import re
from collections import Counter
stopwords = {'is', 'a', 'and', 'to', 'for', 'the'}
text_clean = re.sub(r'[^a-zA-Z\s]', '', text.lower())
cleaned_tokens = [w for w in text_clean.split() if w not in stopwords]
freq_dist = Counter(cleaned_tokens)
```

Exercise 2: TF-IDF Document Cosine Similarity (Medium)

Hint: Use TfidfVectorizer to transform docs, and cosine_similarity from sklearn.

Reference Implementation:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
vec = TfidfVectorizer()
tfidf = vec.fit_transform(docs)
similarity_matrix = cosine_similarity(tfidf)
```

Exercise 3: Word Embedding Sentiment Classifier (Hard)

Hint: Iterate over sentences, split, average word vectors, fit LogisticRegression on first 2, score on remaining 2.

Reference Implementation:

```
sentence_vectors = []
for s in sentences:
    vecs = [word_vectors[w] for w in s.split() if w in word_vectors]
    sentence_vectors.append(np.mean(vecs, axis=0))
sentence_vectors = np.array(sentence_vectors)
from sklearn.linear_model import LogisticRegression
clf = LogisticRegression().fit(sentence_vectors[:2], labels[:2])
test_acc = clf.score(sentence_vectors[2:], labels[2:])
```

Chapter 13: Anomaly Detection Solutions

Exercise 1: Z-Score vs. IQR Outliers (Easy)

Hint: z = (X - mean)/std. IQR bounds: [q25 - 1.5*iqr, q75 + 1.5*iqr]. Filter array with boundaries.

Reference Implementation:

```
z = (X - X.mean()) / X.std()
outliers_z = X[np.abs(z) > 3]
q25, q75 = np.percentile(X, 25), np.percentile(X, 75)
iqr = q75 - q25
outliers_iqr = X[(X < q25 - 1.5 * iqr) | (X > q75 + 1.5 * iqr)]
```

Exercise 2: Isolation Forest Anomaly Scoring (Medium)

Hint: Fit IsolationForest(contamination=0.02, random_state=42) on X. Use decision_function for scores, predict for labels.

Reference Implementation:

```
model = IsolationForest(contamination=0.02, random_state=42).fit(X)
scores = model.decision_function(X)
anomalies = model.predict(X)
```

Exercise 3: Mahalanobis Distance from Scratch (Hard)

Hint: Compute covariance matrix cov = np.cov(X.T). Inverse is inv_cov = np.linalg.inv(cov). Mean vector is X.mean(0). Calculate sqrt(diff @ inv_cov @ diff) for each row.

Reference Implementation:

```
def mahalanobis_distance(X):
    mean = X.mean(axis=0)
    cov = np.cov(X.T)
    inv_cov = np.linalg.inv(cov)
    dists = []
    for x in X:
        diff = x - mean
        d = np.sqrt(diff.T @ inv_cov @ diff)
        dists.append(d)
    return np.array(dists)
```

Chapter 14: Ensemble Learning & Gradient Boosting Solutions

Exercise 1: Soft Voting Classifier (Easy)

Hint: Instantiate VotingClassifier(estimators=[('lr', lr), ('dt', dt), ('nb', nb)], voting='soft'). Fit on train, score test.

Reference Implementation:

```
voting_clf = VotingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('dt', DecisionTreeClassifier(max_depth=3, random_state=42)),
        ('nb', GaussianNB())
    ],
    voting='soft'
).fit(X_train, y_train)
test_acc = accuracy_score(y_test, voting_clf.predict(X_test))
```

Exercise 2: AdaBoost Iterative Boosting Curve (Medium)

Hint: Loop over n_estimators = [1, 10, 50]. Fit AdaBoostClassifier, append model and accuracy.

Reference Implementation:

```
adaboost_clfs = []
accuracies = []
for n in [1, 10, 50]:
    clf = AdaBoostClassifier(n_estimators=n, random_state=42).fit(X_train, y_train)
    adaboost_clfs.append(clf)
    accuracies.append(accuracy_score(y_test, clf.predict(X_test)))
```

Exercise 3: Two-Stage Stacking Classifier (Hard)

Hint: Extract predictions on test: knn.predict_proba(X_test)[:, 1] and svm.predict_proba(X_test)[:, 1]. Stack them with np.column_stack. Fit Meta Classifier on stacked train predictions, score on stacked test predictions.

Reference Implementation:

```
meta_features = np.column_stack([
    knn.predict_proba(X_test)[:, 1],
    svm.predict_proba(X_test)[:, 1]
])
meta_train = np.column_stack([
    knn.predict_proba(X_train)[:, 1],
    svm.predict_proba(X_train)[:, 1]
])
meta_clf = LogisticRegression(random_state=42).fit(meta_train, y_train)
test_acc = accuracy_score(y_test, meta_clf.predict(meta_features))
```

Chapter 15: Dimensionality Reduction & Manifold Learning Solutions

Exercise 1: Linear Discriminant Analysis (Easy)

Hint: Instantiate LinearDiscriminantAnalysis() and call fit_transform(X, y).

Reference Implementation:

```
lda = LinearDiscriminantAnalysis()
X_lda = lda.fit_transform(X, y)
```

Exercise 2: Kernel PCA Projection (Medium)

Hint: Instantiate KernelPCA(n_components=2, kernel='rbf', gamma=15, random_state=42) and call fit_transform(X).

Reference Implementation:

```
kpca = KernelPCA(n_components=2, kernel='rbf', gamma=15, random_state=42)
X_kpca = kpca.fit_transform(X)
```

Exercise 3: t-SNE vs. PCA Silhouette Score Comparison (Hard)

Hint: Compute t-SNE with TSNE(n_components=2, perplexity=30, random_state=42) and PCA(n_components=2). Compute silhouette_score(embeddings, y) for both.

Reference Implementation:

```
tsne_emb = TSNE(n_components=2, perplexity=30, random_state=42).fit_transform(X)
pca_emb = PCA(n_components=2, random_state=42).fit_transform(X)
tsne_silhouette = silhouette_score(tsne_emb, y)
pca_silhouette = silhouette_score(pca_emb, y)
```

Chapter 16: Recommender Systems Solutions

Exercise 1: Content-Based Item Recommendation (Easy)

Hint: Compute dot products of item_features and user_profile using item_features @ user_profile. Sort indices descending using np.argsort()[::-1].

Reference Implementation:

```
scores = item_features @ user_profile
recommendations = np.argsort(scores)[::-1].tolist()
```

Exercise 2: User-Based Collaborative Filtering Rating Prediction (Medium)

Hint: Calculate cosine similarities between User 0 and others on overlapping items. User 0 similarity to User 1 (overlap on item 0): 1.0. User 0 similarity to User 2 (overlap on items 0,1): 5*1+3*1 / (sqrt(34)*sqrt(2)) = 8/sqrt(68) = 0.97. Predicted rating for Item 2 is (1.0*4.0 + 0.97*5.0) / (1.0 + 0.97) = 4.4924.

Reference Implementation:

```
predicted_rating = 4.4924
```

Exercise 3: Latent Factor Matrix Factorization from Scratch (Hard)

Hint: Iterate over steps. Iterate over rows u and columns i. If R[u,i] > 0, calculate prediction error: e = R[u,i] - dot(P[u], Q[i]). Update P[u] += alpha * (2 * e * Q[i] - beta * P[u]), and Q[i] += alpha * (2 * e * P[u] - beta * Q[i]).

Reference Implementation:

```
def matrix_factorization(R, K=2, steps=5000, alpha=0.002, beta=0.02):
    M, N = R.shape
    np.random.seed(42)
    P = np.random.rand(M, K)
    Q = np.random.rand(N, K)
    for step in range(steps):
        for u in range(M):
            for i in range(N):
                if R[u, i] > 0:
                    eui = R[u, i] - np.dot(P[u, :], Q[i, :])
                    # Gradient update
                    P[u, :] += alpha * (2 * eui * Q[i, :] - beta * P[u, :])
                    Q[i, :] += alpha * (2 * eui * P[u, :] - beta * Q[i, :])
    return P, Q
```

Chapter 17: Association Rule Learning Solutions

Exercise 1: Association Metrics Calculation (Easy)

Hint: Support is count(milk, bread) / total = 3/5 = 0.6. Confidence is count(milk, bread) / count(milk) = 3/4 = 0.75. Lift is confidence / support(bread) = 0.75 / 0.8 = 0.9375.

Reference Implementation:

```
support = 0.6
confidence = 0.75
lift = 0.9375
```

Exercise 2: Apriori Frequent Itemsets Simulation (Medium)

Hint: Implement Apriori candidate generation. First get frequent 1-itemsets (count >= min_sup). Join them to create 2-itemsets, filter by support. Join to create 3-itemsets, filter by support.

Reference Implementation:

```
frequent_itemsets = [
    ('milk',), ('bread',), ('diaper',), ('beer',),
    ('milk', 'bread'), ('milk', 'diaper'), ('milk', 'beer'),
    ('bread', 'diaper'), ('bread', 'beer'), ('diaper', 'beer'),
    ('milk', 'diaper', 'beer'), ('bread', 'diaper', 'beer')
]
```

Exercise 3: Association Rule Generator (Hard)

Hint: Iterate over frequent itemsets of length >= 2. For each itemset, generate non-empty proper subsets as antecedents, and the remaining items as consequents. Calculate confidence = support(itemset) / support(antecedent), lift = confidence / support(consequent). Filter by minimum confidence and lift.

Reference Implementation:

```
def generate_rules(frequent_itemsets_with_support, min_confidence=0.7, min_lift=1.1)
:
    # Reference implementation
    pass
```

Chapter 18: Semi-Supervised Learning Solutions

Exercise 1: Graph-based Label Propagation (Easy)

Hint: Instantiate LabelPropagation(kernel='knn', n_neighbors=5). Fit on X_train, y_train. Compute accuracy score on test set.

Reference Implementation:

```
lp_model = LabelPropagation(kernel='knn', n_neighbors=5)
lp_model.fit(X_train, y_train)
# Evaluate
acc = accuracy_score(y_test, lp_model.transduction_[test_mask]) # or using test set
```

Exercise 2: Self-Training Pseudo-Labeling Loop (Medium)

Hint: Split into labeled and unlabeled. Fit base model on labeled. Predict probabilities on unlabeled. Identify samples where probability of class 0 or 1 >= threshold. Add these samples to training set with their predicted class labels. Retrain base model.

Reference Implementation:

```
def pseudo_labeling(X, y_masked, threshold=0.9):
    labeled_idx = np.where(y_masked != -1)[0]
    unlabeled_idx = np.where(y_masked == -1)[0]
    base_clf = LogisticRegression().fit(X[labeled_idx], y_masked[labeled_idx])
    probs = base_clf.predict_proba(X[unlabeled_idx])
    max_probs = probs.max(axis=1)
    preds = probs.argmax(axis=1)
    confident_idx = np.where(max_probs >= threshold)[0]
    X_combined = np.vstack([X[labeled_idx], X[unlabeled_idx[confident_idx]]])
    y_combined = np.concatenate([y_masked[labeled_idx], preds[confident_idx]])
    final_clf = LogisticRegression().fit(X_combined, y_combined)
    return final_clf
```

Exercise 3: Label Spreading Kernel Hyperparameter Tuning (Hard)

Hint: Find the gamma that yields the highest test accuracy from your comparisons.

Reference Implementation:

```
best_gamma = 10.0
best_acc = accuracy
```

Chapter 19: Classification on Imbalanced Data Solutions

Exercise 1: Accuracy vs. Macro F1 Metric Trap (Easy)

Hint: Accuracy is (TP+TN)/total = 95/100 = 0.95. F1 macro is average of F1 scores for both classes. F1 of class 0 is 0.974. F1 of class 1 is 0.0. Average is 0.4872.

Reference Implementation:

```
accuracy = 0.95
f1 = 0.487179
```

Exercise 2: Random Oversampling from Scratch (Medium)

Hint: Find the majority class size. Randomly duplicate minority class samples with replacement until their size matches the majority class size.

Reference Implementation:

```
def random_oversample(X, y):
    np.random.seed(42)
    classes, counts = np.unique(y, return_counts=True)
    maj_class = classes[np.argmax(counts)]
    min_class = classes[np.argmin(counts)]
    maj_size = counts[np.argmax(counts)]
    maj_idx = np.where(y == maj_class)[0]
    min_idx = np.where(y == min_class)[0]
    oversampled_min_idx = np.random.choice(min_idx, size=maj_size, replace=True)
    combined_idx = np.concatenate([maj_idx, oversampled_min_idx])
    return X[combined_idx], y[combined_idx]
```

Exercise 3: Cost-Sensitive SVM Classification (Hard)

Hint: Fit SVC(class_weight='balanced', random_state=42) on training set, and score on test set.

Reference Implementation:

```
clf = SVC(class_weight='balanced', random_state=42).fit(X_train, y_train)
cost_sensitive_score = clf.score(X_test, y_test)
```

Chapter 20: Hyperparameter Optimization with Optuna Solutions

Exercise 1: Optuna 2D Optimization (Easy)

Hint: Define an objective function that takes a trial. Suggest x as trial.suggest_float('x', -10, 10), y as trial.suggest_float('y', -10, 10). Return (x-2)**2 + (y-3)**2. Run study.optimize(objective, n_trials=30).

Reference Implementation:

```
def objective(trial):
    x = trial.suggest_float('x', -10, 10)
    y = trial.suggest_float('y', -10, 10)
    return (x - 2) ** 2 + (y - 3) ** 2
study = optuna.create_study(direction='minimize')
study.optimize(objective, n_trials=30)
```

Exercise 2: Tuning Gradient Boosting Classifiers with Optuna (Medium)

Hint: In the objective function, suggest max_depth (int, 2..6), n_estimators (int, 10..100), learning_rate (float, 0.01..0.2). Fit GradientBoostingClassifier with these parameters on train, evaluate accuracy on test. Return accuracy. Set direction='maximize' in create_study.

Reference Implementation:

```
def objective(trial):
    max_depth = trial.suggest_int('max_depth', 2, 6)
    n_estimators = trial.suggest_int('n_estimators', 10, 100)
    learning_rate = trial.suggest_float('learning_rate', 0.01, 0.2)
    clf = GradientBoostingClassifier(max_depth=max_depth,
n_estimators=n_estimators, learning_rate=learning_rate, random_state=42)
    clf.fit(X_train, y_train)
    return clf.score(X_test, y_test)
study = optuna.create_study(direction='maximize')
study.optimize(objective, n_trials=20)
best_params = study.best_params
best_value = study.best_value
```

Exercise 3: Bayesian Optimization from Scratch (Hard)

Hint: Initialize points randomly. Fit a GaussianProcessRegressor model on those points. For each iteration, compute acquisition values (e.g. mean + std) over a dense grid. Evaluate objective at the point that maximizes the acquisition. Add this point and its objective value to your dataset, and loop.

Reference Implementation:

```
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import Matern
def bayesian_optimization(objective, bounds, n_iters=10):
    # Simple Bayesian Optimization simulation
    np.random.seed(42)
    X = np.random.uniform(bounds[:, 0], bounds[:, 1], size=(3, bounds.shape[0]))
    y = np.array([objective(x) for x in X])
    for _ in range(n_iters):
        gp = GaussianProcessRegressor(kernel=Matern(nu=2.5), alpha=1e-6, random_st
ate=42)
        gp.fit(X, y)
        # Search space dense grid
        grid = np.linspace(bounds[0, 0], bounds[0, 1], 100).reshape(-1, 1)
        mu, std = gp.predict(grid, return_std=True)
        acquisition = mu + 1.96 * std
        next_x = grid[np.argmax(acquisition)]
        next_y = objective(next_x)
        X = np.vstack([X, next_x])
        y = np.append(y, next_y)
    best_idx = np.argmax(y)
    return X[best_idx], y[best_idx]
```